BACSE202 · Database Management Systems · Fall 2026–27 · VIT Chennai

Satellite Imagery Knowledge Graph

GraphRAG for Earth Observation Data

Complete Study Guide

DA1 Design · DA2 SQL & PL/SQL · DA3 Neo4j · Novelty · Web Application · Testing · Viva Question Bank · Glossary

Prepared for Joshwa D Y (25BCE5176) and team · Faculty: Dr. S. Geetha

Contents

How to Use This Guide
The project in 60 seconds

Part 1: Foundations (the basics you need)
1.1 Data, information and databases
1.2 The relational model: tables, rows, columns
1.3 Keys: how rows are identified and linked
1.4 Integrity constraints: rules the database enforces
1.5 SQL and PL/SQL
1.6 Oracle Database 21c XE and SQL Developer
1.7 NoSQL and graph databases (for DA3)
1.8 Earth-observation (EO) basics
1.9 AI basics used in the project
1.10 Web basics (for the application)

Part 2: The Project
2.1 Title and course
2.2 The problem
2.3 Users
2.4 Objectives
2.5 Scope
2.6 The data we used
2.7 Overall architecture
2.8 Repository layout (GitHub)

Part 3: DA1 (Database Design)
3.1 What is an ER model?
3.2 Entities: the "things" (15 of them)
3.3 From 8 entities to 15: why the design grew
3.4 Strong, weak and associative entities
3.5 Attributes and their types
3.6 Relationships, degree, cardinality and participation
3.7 The ER diagram
3.8 Converting the ER model to tables (mapping)
3.9 The relational schema (15 tables)
3.10 Keys summary
3.11 Functional dependencies (FDs)
3.12 Normalization, step by step
3.13 Deliberate denormalization (and why it's justified)
3.14 DA1 viva questions

Part 4: DA2 (SQL and PL/SQL Implementation)
4.1 Setting up Oracle
4.2 Oracle data types we used
4.3 schema.sql: creating the tables
4.4 sample_data.sql: inserting the data
4.5 Transactions and ACID
4.6 SQL query concepts (with our queries)
4.7 PL/SQL basics
4.8 plsql.sql: our programs
4.9 run_procedures.sql: the proof (your actual output)

4.10 extras.sql: extra coverage (your results)
4.11 Problems we hit and fixed (good to mention)
4.12 DA2 viva questions

Part 5: The Novelty (What Makes This Project Special)

5.1 Why novelty was needed
5.2 Novelty 1: Temporal lineage
5.3 Novelty 2: Tunable hybrid retrieval score
5.4 Novelty 3: Confidence propagation and trust-aware alerts
5.5 How the three ideas answer the flagship question together
5.6 Honest limitations of the novelty

Part 6: DA3 (Neo4j Graph Database Extension)

6.1 Why a graph database?
6.2 The property-graph model
6.3 Cypher: the query language
6.4 The graph model (schema)
6.5 Implementation
6.6 The technology-specific queries (03_queries.cypher)
6.7 Database operations (04_crud.cypher)
6.8 Graph views (05_graph_views.cypher)
6.9 The comparison (required)
6.10 DA3 viva questions

Part 7: The Application (Final Project)

7.1 Architecture
7.2 What happens in one request (step by step)
7.3 The 9 features, end to end
7.4 Validation and error handling
7.5 Security
7.6 How to run the app
7.7 How the app uses DA1, DA2 and DA3

Part 8: Testing, Results, Limitations and Future Work

8.1 Three levels of testing
8.2 Key results
8.3 Limitations
8.4 Future work

Part 9: Viva Preparation

9.1 The 2-minute project explanation (memorise the flow, not the words)
9.2 Suggested demo order (3–5 minutes)
9.3 Question bank (cover the answer, try it yourself, then check)

Glossary

How to Use This Guide

This guide explains the **whole project** (DA1, DA2, DA3 and the final application) starting from the very basics. You do not need to know anything beforehand. Every term is explained the first time it appears, and every idea is shown with an example taken from *our* project.

Suggested way to study:

1. **Part 1 (Foundations)**: read this first if words like *table*, *key*, *SQL* or *embedding* are new or fuzzy.
2. **Part 2 (The Project)**: what we built and why, in plain words.
3. **Parts 3–7**: DA1, DA2, the novelty, DA3 and the application, in the same order you will present them.
4. **Part 8**: testing, limitations and future work.
5. **Part 9 (Viva Question Bank)**: practise answering out loud. Cover the answer, try it yourself, then check.
6. **Glossary**: look up any word quickly.

Boxes used in this guide
CONCEPT a definition you must know EXAMPLE how it looks in our project VIVA a likely examiner question CAREFUL a common mistake or subtle point

The project in 60 seconds

Satellites photograph the Earth every day. Each photo comes with information: when it was taken, where, how cloudy it was, which satellite and camera took it, and what an AI model found in it (buildings, ships, patches of forest). Normally these facts are stored in separate places with no links between them, so it is hard to answer connected questions like:

> "*Find satellite images showing **rapid deforestation** near **river basins** in **Q3**, and tell me **how much I can trust** that answer.*"

We built a system that **connects** all this information so such questions can be answered:

- **DA1**: we *designed* the database (ER diagram, 15 tables, normalization to BCNF).
- **DA2**: we *built* it in Oracle with SQL and PL/SQL (tables, constraints, data, 19 queries, a package, triggers).
- **DA3**: we *extended* it with Neo4j, a graph database that is good at connections, map distances and "meaning" search.
- **Final project**: we built a *web application* (Python + Flask) on top of Oracle with 9 working features, and tested it (70 automated tests, 30 test cases).
- **Novelty**: (1) tracking the *same* forest patch over time, (2) ranking evidence by meaning + place + time with adjustable weights, (3) giving every answer a *trust score*.

Part 1: Foundations (the basics you need)

1.1 Data, information and databases

Data is raw facts: `31.4`, `Sentinel-2A`, `2026-09-29`. **Information** is data with meaning: "the image taken by Sentinel-2A on 29 Sep 2026 was 31.4% cloudy".

CONCEPT A **database** is an organised collection of related data, stored so it can be searched, updated and kept correct. A **DBMS (Database Management System)** is the software that manages it, for example Oracle, MySQL, PostgreSQL or Neo4j.

Why not just use Excel or files? Spreadsheets break down when:

- **many people** change data at the same time (a DBMS handles this safely with *transactions*);
- you need **rules** ("cloud cover must be 0–100") that are enforced every time;
- the data is **linked** (an image belongs to a sensor, which belongs to a satellite);
- the data is **large** and must be searched quickly (a DBMS uses *indexes*);
- you must not lose data in a crash (a DBMS has logs and recovery).

1.2 The relational model: tables, rows, columns

The most common kind of database is **relational**. It stores data in **tables** (formally called *relations*).

Term	Also called	Meaning	Our example
Table	Relation	a set of similar records	SATELLITE
Row	Tuple, record	one record	the row for "Sentinel-2A"
Column	Attribute, field	one property of every record	launch_date
Domain	Data type	the allowed values of a column	dates; numbers 0–100
Schema	Structure	the design: which tables and columns exist	our 15 tables

EXAMPLE Part of our SATELLITE table:

satellite_id	satellite_name	operator_name	launch_date	orbit_type
1	Sentinel-2A	ESA/Copernicus	2015-06-23	SUN-SYNCHRONOUS
3	Landsat 8	NASA/USGS	2013-02-11	SUN-SYNCHRONOUS

1.3 Keys: how rows are identified and linked

CONCEPT **Primary key (PK)**: the column (or columns) that uniquely identifies each row. It can never be empty or repeated. *Example:* `satellite_id`.
Candidate key: any column (or set of columns) that *could* serve as the primary key because it is also unique. *Example:* `satellite_name` is unique too.
Foreign key (FK): a column that points to the primary key of another table, which is how tables are linked. *Example:* `sensor.satellite_id` points to `satellite.satellite_id`.
Composite key: a key made of more than one column. *Example:* (image_id, index_name).
Surrogate key: an artificial number used as the PK instead of a natural value. *Example:* `image_id` instead of the long `file_path`.
Partial key: in a weak entity, the attribute that is unique only *within one owner*. *Example:* `sensor_name`, since "MSI" is unique only within one satellite.

Why surrogate keys? Natural values can be long (a file path), can change (a name can be corrected) or can be made of several columns. A small number that never changes makes links fast and simple. We still keep the natural values **UNIQUE**, so no information is lost.

1.4 Integrity constraints: rules the database enforces

A **constraint** is a rule the DBMS checks on every insert and update. If the rule is broken, the change is rejected with an error.

- **Entity integrity**: every row has a non-empty, unique primary key.
- **Referential integrity**: a foreign key must point to a row that exists. You cannot have an image that belongs to sensor 999 if sensor 999 doesn't exist.
- **Domain integrity**: values must be of the right type and range (CHECK constraints such as `cloud_cover BETWEEN 0 AND 100`).
- **Uniqueness**: UNIQUE constraints, e.g. no two satellites with the same name.
- **Not null**: a required value must be present.

1.5 SQL and PL/SQL

CONCEPT **SQL (Structured Query Language)** is the standard language for relational databases. It is *declarative*: you say **what** you want, not **how** to get it. SQL statements fall into groups:

Group	Purpose	Commands
DDL: Data Definition Language	create or change structure	CREATE, ALTER, DROP, COMMENT
DML: Data Manipulation Language	change data	INSERT, UPDATE, DELETE, MERGE
DQL: Data Query Language	read data	SELECT
DCL: Data Control Language	permissions	GRANT, REVOKE
TCL: Transaction Control Language	save or undo	COMMIT, ROLLBACK, SAVEPOINT

PL/SQL is Oracle's *procedural* extension of SQL. It adds variables, IF/ELSE, loops, error handling, functions, procedures, packages and triggers. You use it when you need logic that runs step by step, automatically, or repeatedly. Examples: "for every forest patch, compare it with the previous one", or "whenever a detection is inserted, write an audit row".

1.6 Oracle Database 21c XE and SQL Developer

- **Oracle Database 21c Express Edition (XE)** is the free version of Oracle that we used. It contains a *container database* called XE with a *pluggable database* called **XEPDB1**, where our work lives.
- In Oracle, a **user** and a **schema** are tied together: user **JOSHWA** owns the schema JOSHWA, which holds all our tables. "Creating a database" for a project in Oracle really means creating a user/schema.
- **SQL Developer** is the graphical tool we used to connect (host `localhost`, port `1521`, service `XEPDB1`) and run scripts. **F5** runs a whole script; **Ctrl+Enter** runs one statement and shows a result grid.
- **SQL*Plus** is Oracle's command-line tool. We used it once, as the administrator, to grant privileges.

1.7 NoSQL and graph databases (for DA3)

"NoSQL" databases store data in forms other than tables. The main families:

Family	Stores data as	Example products
Document	JSON-like documents	MongoDB
Key–value	key → value pairs	Redis
Column-family	wide rows grouped in column families	Cassandra
Graph	nodes and relationships	**Neo4j**
Vector	number vectors for similarity search	Milvus, Qdrant
Time-series	timestamped measurements	InfluxDB

A **graph database** stores **nodes** (things) and **relationships** (named links between things). Links are stored directly, so following them is fast. This suits highly connected data like ours. Details are in Part 6.

1.8 Earth-observation (EO) basics

- **Earth observation**: gathering information about the Earth from satellites, aircraft or drones.
- **Satellite**: the spacecraft (e.g. Sentinel-2A by ESA, Landsat 8 by NASA/USGS).
- **Sensor**: the instrument (camera) on the satellite. Sentinel-2's **MSI** sees 13 colour bands at 10 m resolution; Landsat 8's **OLI** sees 9 bands at 30 m.
- **Spatial resolution**: the size of one pixel on the ground. 10 m means each pixel covers 10 m × 10 m. Smaller numbers mean more detail.

- **Orbit**: **sun-synchronous** satellites pass over each place at the same local time, so lighting is consistent between images.
- **Revisit time**: how often the satellite photographs the same place (Sentinel-2 about 5–10 days, Landsat 16 days).
- **Cloud cover**: the percentage of the image hidden by clouds. Optical cameras can't see through clouds, so cloudy images are less reliable.
- **Sun elevation**: the sun's angle above the horizon when the image was taken. Low angles give long shadows.
- **Bounding box (bbox)**: the smallest rectangle around a detected object, given by its minimum and maximum latitude and longitude.
- **Object detection**: an AI model draws bounding boxes around things it recognises and gives each a **confidence** between 0 and 1 (0.91 means "91% sure").

Spectral bands and indices (NDVI, NDWI)

Sensors measure reflected sunlight in several **bands** (colours), including **near-infrared (NIR)**, which our eyes can't see. Healthy plants reflect a lot of NIR and absorb red light; water absorbs NIR. A **spectral index** combines two bands into one number between −1 and +1:

Index	Formula	Shows	Typical values
NDVI	(NIR − Red) / (NIR + Red)	vegetation greenness	forest 0.6–0.9; bare soil 0.1–0.2; water < 0
NDWI	(Green − NIR) / (Green + NIR)	water	water > 0.2; land < 0

When forest is cut, NDVI drops. We store each image's **mean, min and max** NDVI and NDWI.

River basin

A **river basin** (catchment) is all the land whose rain and streams drain into one river. The Amazon basin covers about 6 million km². Forests near rivers protect them from erosion and floods, which is why "deforestation near river basins" matters. In our database each location may *belong to* a basin, and we measure its distance to a **reference point on the main river**.

1.9 AI basics used in the project

CONCEPT An **embedding** is a list of numbers (a *vector*) produced by an AI model so that **things with similar meaning get similar numbers**. A photo of a cleared forest and another cleared forest end up with vectors pointing in similar directions. In our project each embedding has **8 numbers** (real systems use 512 or more).

Cosine similarity measures how alike two vectors are by the *angle* between them:

```
cos(A, B) = (A · B) / (|A| × |B|)
```

1 means the same direction (very similar), 0 means unrelated, −1 means opposite.

WORKED EXAMPLE A = (1, 0), B = (1, 1).
A·B = 1×1 + 0×1 = 1. |A| = 1, |B| = √2 ≈ 1.414. cos = 1 / 1.414 ≈ **0.707**, so they are fairly similar (an angle of 45°).
A = (1, 0), C = (0, 1): A·C = 0, so cos = **0**, unrelated (an angle of 90°).

RAG (Retrieval-Augmented Generation): before an AI answers a question, it first *retrieves* relevant facts from a data store and bases the answer on them. **GraphRAG** does the retrieval using a **graph** (following connections) as well as similarity search. Our system does the retrieval and the trust scoring. Generating free-text answers with a language model is future work.

1.10 Web basics (for the application)

- **Client-server**: your browser (client) asks; a program (server) answers.
- **HTTP**: the protocol for asking and answering. **GET** asks for a page (e.g. a search); **POST** sends data that changes something (e.g. a form).
- **URL**: the address, e.g. `http://127.0.0.1:5000/finder?max_km=300`. `127.0.0.1` means "this computer", 5000 is the port, and the part after `?` is the parameters.
- **HTML** = page content, **CSS** = style, **JavaScript** = interactive behaviour (maps, charts).
- **Flask**: a small Python framework that maps URLs to Python functions.

Part 2: The Project

2.1 Title and course

Satellite Imagery Knowledge Graph: GraphRAG for Earth Observation Data. BACSE202 Database Management Systems, Fall Semester 2026–27, VIT Chennai. Faculty: Dr. S. Geetha. The same project continues through DA1 (design), DA2 (SQL/PL-SQL), DA3 (modern database) and the final submission (application + report + video).

2.2 The problem

EO data is **fragmented**. For each satellite image there are:

- image metadata: time, place, cloud cover, file path;
- instrument details: satellite, sensor, resolution;
- AI outputs: objects detected, their confidence and size;
- derived measures: NDVI, NDWI.

These usually live in different files and tables with **no links**, and with no record that two detections are the **same physical object** seen at different times. So connected questions are hard. Our driving question combines four kinds of reasoning:

Part of the question	Kind of reasoning	How we answer it
"deforestation"	meaning (semantic)	tree-patch type + embedding similarity
"rapid"	change over time	EVOLVED_FROM lineage + loss rate %/month
"near river basins"	space	distance to the basin's river point
"in Q3"	time	capture_time between 1 Jul and 30 Sep
"how much to trust it"	uncertainty	detection confidence × clear-sky fraction

2.3 Users

User	Needs
EO analyst	search images, inspect locations, ask questions, review alerts
Data/ML engineer	load images and detections, register AI models
Administrator	manage satellites, users and the audit trail

2.4 Objectives

1. Design a normalised (BCNF) database linking satellites, sensors, images, locations, basins and detected objects.
2. Track the same real-world object across time so change can be measured.
3. Log questions with their ranked evidence and an answer-confidence score.
4. Raise alerts only for trustworthy changes; audit every change to detections.
5. Extend the system with a graph database for connected, spatial and semantic queries.

6. Provide a working, tested web application on top of the database.

2.5 Scope

In scope: image metadata, detected objects, spectral indices, change tracking, alerts, question/evidence logs, auditing. **Out of scope:** storing raw image pixels (we store only the file path), live satellite downlink, training AI models, and user login.

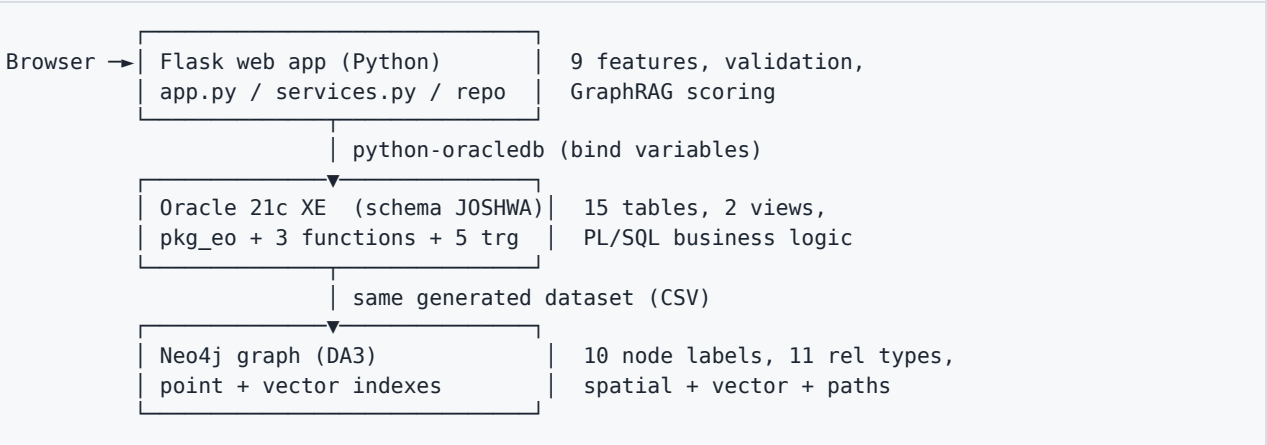
2.6 The data we used

All data is **synthetic** (realistic but made up), as the brief requires. It is produced by `scripts/generate_data.py`:

- 5 satellites (Sentinel-2A/2B, Landsat 8/9, Sentinel-1A), 6 sensors, 5 river basins (Amazon, Congo, Mekong, Godavari, Danube), 12 locations (9 inside basins and 3 outside: Chennai Coast, Rotterdam Port, Dubai Marina).
- 5 capture dates (10 Apr, 18 Jun, 22 Jul, 27 Aug, 29 Sep 2026), giving 12 × 5 = **60 images**.
- **247 detected objects** of 8 types; **120 spectral indices**; 4 users; 4 sample questions with 20 evidence rows.
- Designed patterns: **rapid** deforestation at Manaus, Kisangani, Tonle Sap and Bhadrachalam; **slow** loss at Rondonia, Kratie and Rajahmundry; **stable** at Mbandaka and Belgrade.

Why a generator script? It uses a fixed random seed (2026), so it produces **identical data every time**. The **same data** feeds both Oracle (SQL inserts) and Neo4j (CSV files), so their answers can be compared. It also writes `expected_results.txt`, which we used to check that the PL/SQL calculations were correct (they matched to 4 decimal places).

2.7 Overall architecture



Oracle is the system of record: it holds the authoritative data with strict rules. **Neo4j is the retrieval layer**: it makes connected, spatial and semantic questions easy. **The app** lets users work with the data.

2.8 Repository layout (GitHub)

Folder / file	Contents
database/	schema.sql, sample_data.sql, plsql.sql, run_procedures.sql, queries.sql, extras.sql
src/	the Flask app: app.py, services.py, repo.py, db.py, config.py, templates/, static/
tests/	unit, UI and live-database tests; TEST_CASES.md; expected_results.txt
neo4j/	Cypher scripts 01–05, CSV import files, graph model diagram
scripts/	generate_data.py
docs/	DA1/DA2/DA3 content, ER and schema diagrams
screenshots/, report/, presentation/	evidence, final report PDF, slides
README.md, requirements.txt, .env.example, .gitignore	documentation, Python packages, configuration template, files excluded from Git

Part 3: DA1 (Database Design)

What DA1 asked for: an ER/EER diagram, conversion of the ER model into relational tables, normalization up to BCNF where applicable, a handwritten report and a presentation. DA1 is the **blueprint**: nothing is built yet, but every later step depends on it.

3.1 What is an ER model?

CONCEPT The **Entity-Relationship (ER) model**, introduced by Peter Chen in 1976, describes a system as **entities** (things), their **attributes** (properties) and the **relationships** between them. It is drawn as an **ER diagram** before any table is created. **EER (Enhanced ER)** adds ideas like specialization and generalization.

Chen notation (the one we draw):

Shape	Means
Rectangle	strong entity
Double rectangle	weak entity
Diamond	relationship
Double diamond	identifying relationship (links a weak entity to its owner)
Oval	attribute (underlined = key; dashed underline = partial key; double oval = multi-valued; dashed oval = derived)
Single line	partial participation
Double line	total participation
1, N, M labels	cardinality

3.2 Entities: the "things" (15 of them)

CONCEPT An **entity** is a real-world thing we store data about. An **entity type** is the category (SATELLITE); an **entity instance** is one specific thing (Sentinel-2A).

Remember them as a story, group by group.

Group A: How an image is captured

- **SATELLITE**: the spacecraft. Attributes: name, operator, launch date, orbit type, status, revisit days.
- **SENSOR**: the camera on a satellite. It is separate because **resolution and band count belong to the camera**, and one satellite can carry several cameras (Landsat 8 carries OLI at 30 m and TIRS at 100 m).

- **IMAGE_FRAME**: one captured scene. Attributes: capture time, cloud cover, sun elevation, file path, embedding.

Group B: Where

- **LOCATION**: a monitored place (Manaus Forest Edge), with country, coordinates and region type (FOREST, PORT…).
- **RIVER_BASIN**: the large river region a location may lie in (Amazon Basin), with a code, area and a river reference point.

Group C: What the AI found

- **DETECTED_OBJECT**: one specific object found in one image (this tree patch, this ship), with bounding box, confidence, area, embedding and extra attributes.
- **OBJECT_TYPE**: the general category (Building, Vessel, Tree Coverage…). There are 8 types and 247 objects. *Analogy:* "Car" in a dictionary vs "the red car outside your house".
- **DETECTION_MODEL**: which AI model and version made the detection (ForestNet 1.2, YOLO-EO 8.0, NDVI-Threshold 2.0).
- **SPECTRAL_INDEX**: NDVI and NDWI summary values for an image.

Group D: The novelty and operations

- **OBJECT_CHANGE**: links the **same physical object in two images** and records the change (EVOLVED_FROM).
- **ALERT**: raised for a significant, trustworthy change; it can be assigned and resolved.
- **APP_USER**: the people using the system (analyst, admin, viewer).
- **QUERY_LOG**: a question asked, with weights α/β/γ, the response and an answer confidence.
- **QUERY_EVIDENCE**: which objects were used as evidence for a question, with their scores and rank.
- **AUDIT_LOG**: an automatic history of changes, written only by triggers.

3.3 From 8 entities to 15: why the design grew

The team's first ER diagram had 8 entities (Satellite, Image_Frame, Location, Detected_Object, Object_Type, River_Basin, Query_Log, Query_Evidence). During review and normalization we changed:

Change	Reason
Added SENSOR	resolution depends on the camera, not the image. Storing it on the image was a transitive dependency (image → sensor → resolution)
Removed satellite_id and resolution from IMAGE_FRAME	both follow from sensor_id
Removed basin_id from DETECTED_OBJECT	object → image → location → basin already gives it; storing it twice risked contradictions

Added DETECTION_MODEL	replaces a free-text "model_version" repeated on every object
EVOLVED_FROM became OBJECT_CHANGE	the change has its own data, and ALERT needs to point at a change
Added SPECTRAL_INDEX, APP_USER, ALERT, AUDIT_LOG	needed for vegetation evidence, users, alerting and auditing
latitude/longitude and the 4 bbox columns instead of a geometry type	Oracle XE has no spatial geometry type; plain numbers can be checked and indexed

VIVA **"Why 15 tables?"** "While normalizing, we separated SENSOR and DETECTION_MODEL to remove transitive dependencies. EVOLVED_FROM became OBJECT_CHANGE because the change has its own attributes and alerts must reference it. And we added supporting tables for spectral evidence, users, alerts and auditing."

3.4 Strong, weak and associative entities

CONCEPT **Strong entity**: has its own key and exists independently.
Weak entity: cannot be uniquely identified by its own attributes. It is identified by its **owner's key + its partial key**, and it cannot exist without its owner. Drawn as a double rectangle and linked by a double diamond.
Associative entity: a relationship that is turned into an entity because it has its own attributes, or because other things must relate to it. It usually resolves an M:N relationship.

Entity	Type	Why
SATELLITE, RIVER_BASIN, LOCATION, IMAGE_FRAME, OBJECT_TYPE, DETECTION_MODEL, APP_USER, QUERY_LOG, ALERT, AUDIT_LOG	Strong	own identity, exist independently
SENSOR	**Weak**	"MSI" alone is not unique, because Sentinel-2A and 2B both have an MSI. Owner = SATELLITE, partial key = **sensor_name**. Identifying relationship = CARRIES
SPECTRAL_INDEX	**Weak**	"NDVI" alone is not unique, because every image has one. Owner = IMAGE_FRAME, partial key = **index_name**. Identifying relationship = HAS_INDEX

DETECTED_OBJECT	Strong, existence-dependent	cannot exist without its image, but has no natural partial key (two buildings in one image have no name to tell them apart), so it is given its own surrogate key
OBJECT_CHANGE	**Associative**	the recursive relationship EVOLVED_FROM (object → object) promoted to an entity: it has attributes, and ALERT relates to it
QUERY_EVIDENCE	**Associative**	resolves the M:N between QUERY_LOG and DETECTED_OBJECT; carries the scores and rank

EXAMPLE How (image_id, index_name) identifies a spectral index:

image_id	index_name	mean_value
1	NDVI	0.79
1	NDWI	0.12
2	NDVI	0.74

"NDVI" matches 60 rows and "image 1" matches 2, but "image 1 + NDVI" matches exactly one row.

CAREFUL In the database we gave SENSOR and SPECTRAL_INDEX surrogate IDs (sensor_id, index_id) for convenience. They are still weak entities, because the rules are kept: **UNIQUE(satellite_id, sensor_name)** and **UNIQUE(image_id, index_name)** preserve the identity, and **ON DELETE CASCADE** from the owner preserves the existence dependency. Say this clearly in the viva.

3.5 Attributes and their types

Type	Meaning	Our examples
Simple (atomic)	cannot be split meaningfully	cloud_cover, launch_date, type_name
Composite	made of meaningful parts	**bbox** = (min_lat, min_lon, max_lat, max_lon); **position** = (latitude, longitude); **river reference point** = (lat, lon); **retrieval weights** = (α, β, γ)
Single-valued	one value per entity	almost all attributes
Multi-valued	several values per entity	**embedding** (a list of 8 numbers); **attributes_json** (a varying set of properties, e.g. floors and material for a building, class and length for a vessel)
Derived	can be computed from other stored data	change_pct, time_delta_days, rate_pct_per_month, change_type, severity, retrieval_score, aggregated_confidence

Key	identifies the entity	satellite_id; candidate keys satellite_name, basin_code, email, file_path
Partial key	unique only within the owner	sensor_name, index_name

CAREFUL **confidence_score** is produced by the AI model, not calculated from other database columns, so from the database's point of view it is a *stored* simple attribute. The embeddings are "derived" in the sense that a model computed them from pixels.

3.6 Relationships, degree, cardinality and participation

CONCEPT **Relationship**: an association between entities (a diamond in the diagram).
Degree: how many entity types take part. Binary = 2. Unary (recursive) = 1, where an entity relates to itself.
Cardinality ratio: the maximum number of instances on each side: 1:1, 1:N or M:N.
Participation: the minimum. **Total** (every instance *must* take part, a double line) or **partial** (it *may*, a single line).

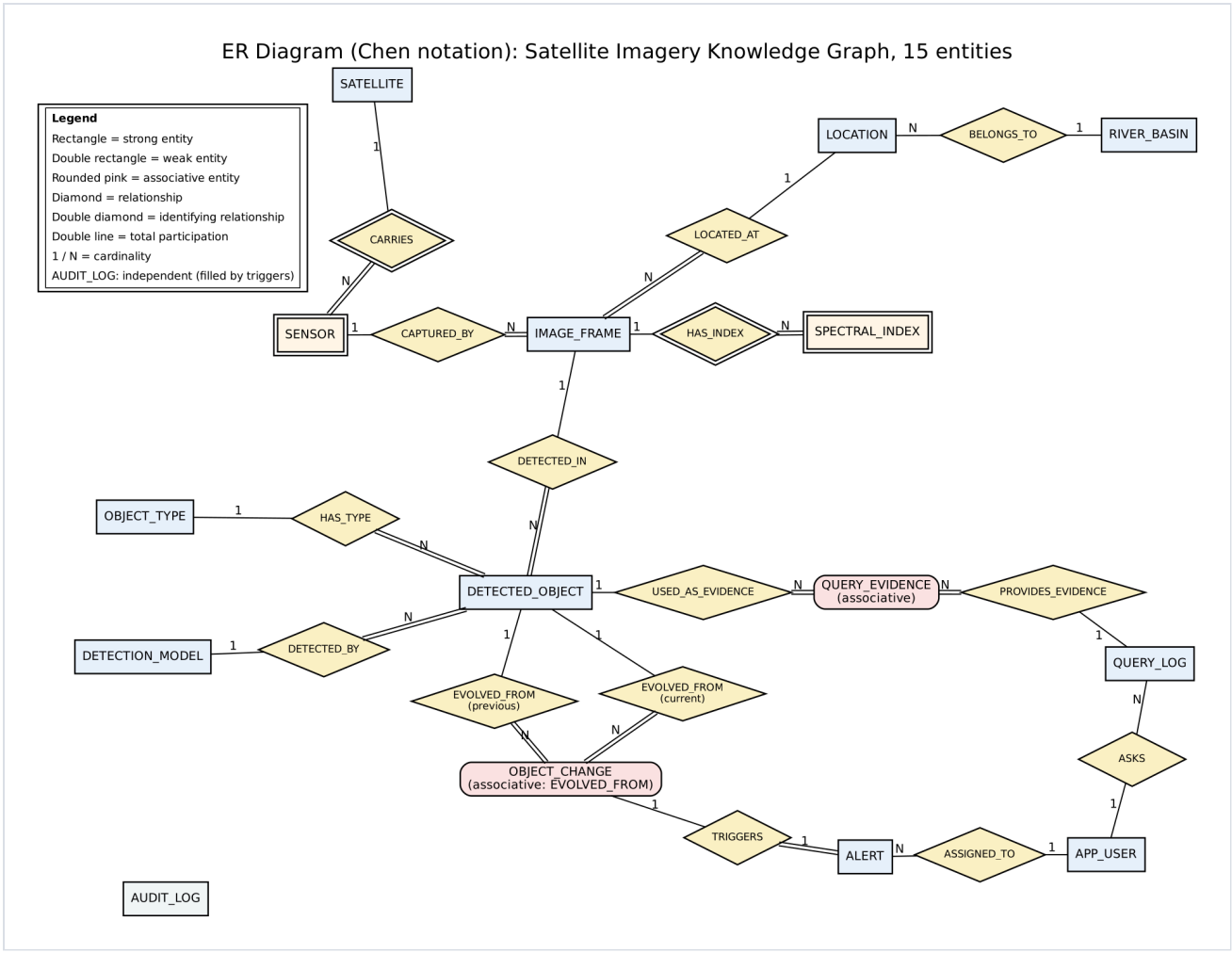
Relationship	Between	Degree	Card.	Participation
CARRIES (identifying)	SATELLITE – SENSOR	binary	1:N	satellite partial; sensor **total**
CAPTURED_BY	SENSOR – IMAGE_FRAME	binary	1:N	sensor partial; image **total**
LOCATED_AT	LOCATION – IMAGE_FRAME	binary	1:N	location partial; image **total**
BELONGS_TO	RIVER_BASIN – LOCATION	binary	1:N	both partial (Chennai Coast is in no basin)
HAS_INDEX (identifying)	IMAGE_FRAME – SPECTRAL_INDEX	binary	1:N	image partial; index **total**
DETECTED_IN	IMAGE_FRAME – DETECTED_OBJECT	binary	1:N	image partial (an image may contain nothing); object **total**
HAS_TYPE	OBJECT_TYPE – DETECTED_OBJECT	binary	1:N	type partial; object **total**
DETECTED_BY	DETECTION_MODEL – DETECTED_OBJECT	binary	1:N	model partial; object **total**
EVOLVED_FROM (via OBJECT_CHANGE)	DETECTED_OBJECT – DETECTED_OBJECT	**unary (recursive)**, roles "previous" and "current"	1:N per role	object partial (the first sighting has no predecessor); change **total** to both

Evidence (via QUERY_EVIDENCE)	QUERY_LOG – DETECTED_OBJECT	binary	**M:N**	both partial; evidence row **total** to both
ASKS	APP_USER – QUERY_LOG	binary	1:N	both partial (anonymous questions allowed)
TRIGGERS	OBJECT_CHANGE – ALERT	binary	**1:1**	change partial (only significant ones); alert **total**
ASSIGNED_TO	APP_USER – ALERT	binary	1:N	both partial (an alert may be unassigned)

Rule to remember: total participation becomes a **NOT NULL** foreign key; partial participation becomes a **nullable** foreign key. For example, every image *must* have a location, so `image_frame.location_id` is NOT NULL. A location *may* have a basin, so `location.basin_id` is nullable.

3.7 The ER diagram

Draw this diagram in the report. Rectangles are entities, double rectangles weak entities, rounded pink boxes associative entities, diamonds relationships; double lines show total participation.



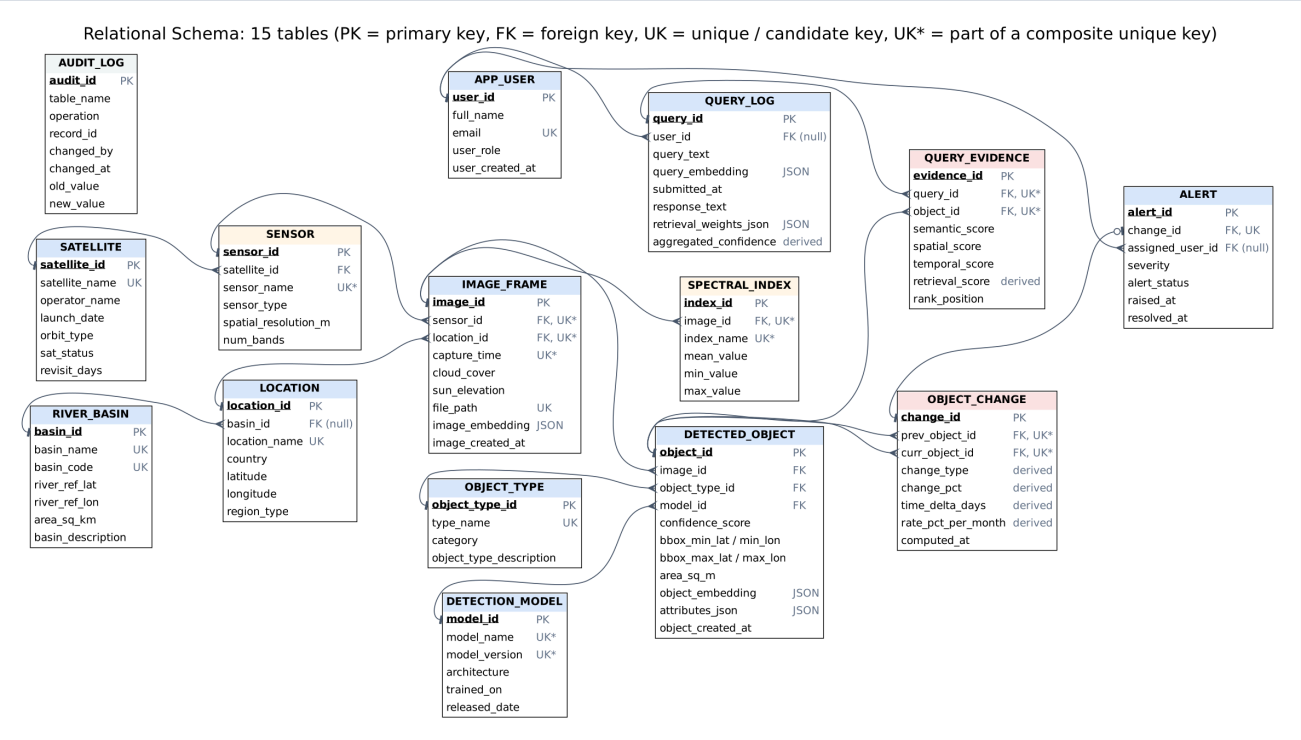
3.8 Converting the ER model to tables (mapping)

Step	ER element	Becomes	Our example
1	Strong entity	a table with the key as the PK	SATELLITE(satellite_id PK, …)
2	Weak entity	a table containing the owner's PK as an FK + the partial key; (FK, partial key) unique; CASCADE on delete	SENSOR(…, satellite_id FK, sensor_name), UNIQUE(satellite_id, sensor_name)
3	1:N relationship	an FK on the **N side**	DETECTED_OBJECT.image_id → IMAGE_FRAME
4	1:1 relationship	an FK on the total side + UNIQUE	ALERT.change_id UNIQUE
5	M:N relationship	a separate bridge table with two FKs	QUERY_EVIDENCE(query_id FK, object_id FK)
6	Recursive relationship with attributes	a table with two FKs to the same table	OBJECT_CHANGE(prev_object_id, curr_object_id)

7	Composite attribute	one column per component	bbox → 4 columns
8	Multi-valued attribute	normally a child table; we chose a JSON column (justified in 3.13)	object_embedding (CLOB, IS JSON)

WHY THE FK GOES ON THE "MANY" SIDE One image has many objects. If we put object IDs in the image row, we would need a list (which is not allowed in a column). If we put `image_id` in each object row, each object stores exactly one value. So the "many" side always holds the foreign key.

3.9 The relational schema (15 tables)



```
SATELLITE      (satellite_id PK, satellite_name UK, operator_name, launch_date, orbit_type,
sat_status, revisit_days)
SENSOR         (sensor_id PK, satellite_id FK, sensor_name, sensor_type, spatial_resolution_m,
num_bands)
                UNIQUE(satellite_id, sensor_name)
RIVER_BASIN    (basin_id PK, basin_name UK, basin_code UK, river_ref_lat, river_ref_lon,
area_sq_km, basin_description)
LOCATION       (location_id PK, basin_id FK NULL, location_name UK, country, latitude, longitude,
region_type)
IMAGE_FRAME    (image_id PK, sensor_id FK, location_id FK, capture_time, cloud_cover,
sun_elevation,
                file_path UK, image_embedding, image_created_at)   UNIQUE(sensor_id, location_id,
capture_time)
OBJECT_TYPE    (object_type_id PK, type_name UK, category, object_type_description)
DETECTION_MODEL(model_id PK, model_name, model_version, architecture, trained_on, released_date)
                UNIQUE(model_name, model_version)
DETECTED_OBJECT(object_id PK, image_id FK, object_type_id FK, model_id FK, confidence_score,
                bbox_min_lat, bbox_min_lon, bbox_max_lat, bbox_max_lon, area_sq_m,
                object_embedding, attributes_json, object_created_at)
OBJECT_CHANGE  (change_id PK, prev_object_id FK, curr_object_id FK, change_type, change_pct,
                time_delta_days, rate_pct_per_month, computed_at)  UNIQUE(prev_object_id,
curr_object_id)
SPECTRAL_INDEX (index_id PK, image_id FK, index_name, mean_value, min_value, max_value)
                UNIQUE(image_id, index_name)
APP_USER       (user_id PK, full_name, email UK, user_role, user_created_at)
QUERY_LOG      (query_id PK, user_id FK NULL, query_text, query_embedding, submitted_at,
response_text,
                retrieval_weights_json, aggregated_confidence)
QUERY_EVIDENCE (evidence_id PK, query_id FK, object_id FK, semantic_score, spatial_score,
temporal_score,
                retrieval_score, rank_position)                     UNIQUE(query_id, object_id)
ALERT          (alert_id PK, change_id FK UK, assigned_user_id FK NULL, severity, alert_status,
raised_at, resolved_at)
AUDIT_LOG      (audit_id PK, table_name, operation, record_id, changed_by, changed_at, old_value,
new_value)
```

3.10 Keys summary

Table	PK	Candidate / unique keys	Foreign keys	Partial key
SATELLITE	satellite_id	satellite_name	none	none
SENSOR	sensor_id	(satellite_id, sensor_name)	satellite_id	**sensor_name**
RIVER_BASIN	basin_id	basin_code; basin_name	none	none
LOCATION	location_id	location_name	basin_id (nullable)	none
IMAGE_FRAME	image_id	file_path; (sensor_id, location_id, capture_time)	sensor_id, location_id	none
OBJECT_TYPE	object_type_id	type_name	none	none
DETECTION_MODEL	model_id	(model_name, model_version)	none	none
DETECTED_OBJECT	object_id	none	image_id, object_type_id, model_id	none
OBJECT_CHANGE	change_id	(prev_object_id, curr_object_id)	prev_object_id, curr_object_id	none

SPECTRAL_INDEX	index_id	(image_id, index_name)	image_id	**index_name**
APP_USER	user_id	email	none	none
QUERY_LOG	query_id	none	user_id (nullable)	none
QUERY_EVIDENCE	evidence_id	(query_id, object_id)	query_id, object_id	none
ALERT	alert_id	change_id	change_id, assigned_user_id (nullable)	none
AUDIT_LOG	audit_id	none	none	none

3.11 Functional dependencies (FDs)

CONCEPT A **functional dependency** X → Y means: **if two rows have the same X, they must have the same Y**. In other words, X determines Y. X is the **determinant**. Example: `satellite_name → operator_name`. Once you know the satellite, its operator is fixed.

Kinds of FD you must recognise:

- **Trivial**: Y is part of X, e.g. (a, b) → a. Always true and ignored.
- **Full**: Y depends on the *whole* composite key, not just part of it.
- **Partial**: Y depends on *part* of a composite key. This violates 2NF.
- **Transitive**: X → Z → Y, where Z is not a key. This violates 3NF.

The FDs of our domain:

#	Functional dependency
FD1	satellite_name → operator_name, launch_date, orbit_type, sat_status, revisit_days
FD2	(satellite_name, sensor_name) → sensor_type, spatial_resolution_m, num_bands
FD3	file_path → capture_time, cloud_cover, sun_elevation, satellite_name, sensor_name, location_name
FD4	(satellite_name, sensor_name, location_name, capture_time) → file_path
FD5	location_name → country, latitude, longitude, region_type, basin_code
FD6	basin_code → basin_name, river_ref_lat, river_ref_lon, area_sq_km; and basin_name → basin_code
FD7	type_name → category, object_type_description
FD8	(model_name, model_version) → architecture, trained_on, released_date
FD9	(file_path, object_no) → type_name, model_name, model_version, confidence, bbox…, area_sq_m, attributes
FD10	(file_path, index_name) → mean_value, min_value, max_value

3.12 Normalization, step by step

Why normalize? Anomalies

Imagine all the data in one big flat table where every row repeats the satellite's operator, the location's country and the basin's area. Three problems appear:

- **Update anomaly**: changing the Amazon basin's area means editing hundreds of rows. Miss one and the database contradicts itself.
- **Insert anomaly**: you cannot record a new satellite until it has taken an image (there is no row to put it in).
- **Delete anomaly**: deleting Chennai's last image also deletes Chennai's coordinates and country.

Normalization splits tables so that each fact is stored once. The normal forms are a ladder; each step removes one kind of bad dependency.

Step 0: the unnormalised relation (UNF)

```
EO_RECORD( file_path, capture_time, cloud_cover, sun_elevation,
           satellite_name, operator_name, launch_date, orbit_type, ...,
           sensor_name, sensor_type, spatial_resolution_m, num_bands,
           location_name, country, latitude, longitude, region_type,
           basin_code, basin_name, river_ref_lat, river_ref_lon, area_sq_km,
           { object_no, type_name, category, model_name, model_version, architecture,
             confidence, bbox, area_sq_m, attributes }*,      <- repeating group 1
           { index_name, mean_value, min_value, max_value }* ) <- repeating group 2
```

Problems: **repeating groups** (many objects and indices inside one image row), a **non-atomic** bbox, and massive redundancy.

Step 1: First Normal Form (1NF)

1NF RULE Every cell holds a single, atomic value, and there are no repeating groups.

Fix: split bbox into 4 columns, and give each object and each index its own row:

```
R1_OBJECT( file_path, object_no, <all image, satellite, sensor, location, basin columns>,
           type_name, category, model_name, model_version, architecture,
           confidence, bbox_min_lat, bbox_min_lon, bbox_max_lat, bbox_max_lon, area_sq_m,
attributes )
           Key = (file_path, object_no)
R1_INDEX ( file_path, index_name, mean_value, min_value, max_value, <image columns> )
           Key = (file_path, index_name)
```

Step 2: Second Normal Form (2NF)

2NF RULE 1NF, and no non-key attribute depends on only **part** of a composite key (no partial dependencies).

In R1_OBJECT the key is (file_path, object_no), but cloud_cover, satellite_name, location_name and so on depend only on **file_path**. That is a partial dependency. Fix: move the image columns into their own table.

```
R2_IMAGE  ( file_path PK, capture_time, cloud_cover, sun_elevation,
            satellite_name, operator_name, ..., sensor_name, spatial_resolution_m, ...,
            location_name, country, ..., basin_code, basin_name, ... )
R2_OBJECT ( file_path FK, object_no, type_name, category, model_name, model_version,
architecture,
            confidence, bbox..., area_sq_m, attributes )            Key = (file_path, object_no)
R2_INDEX  ( file_path FK, index_name, mean_value, min_value, max_value ) Key = (file_path,
index_name)
```

Step 3: Third Normal Form (3NF)

3NF RULE 2NF, and no **transitive dependency**: a non-key attribute must not depend on another non-key attribute.

Transitive chains still present:

- file_path → satellite_name → operator_name … (FD1)
- file_path → (satellite_name, sensor_name) → spatial_resolution_m (FD2)
- file_path → location_name → country … → basin_code → basin_name (FD5, FD6)
- (file_path, object_no) → type_name → category (FD7)
- (file_path, object_no) → (model_name, model_version) → architecture (FD8)

Fix: give each its own table.

```
SATELLITE       ( satellite_name PK, operator_name, launch_date, orbit_type, sat_status,
revisit_days )
SENSOR          ( satellite_name FK, sensor_name, sensor_type, spatial_resolution_m, num_bands )
RIVER_BASIN     ( basin_code PK, basin_name, river_ref_lat, river_ref_lon, area_sq_km )
LOCATION        ( location_name PK, country, latitude, longitude, region_type, basin_code FK )
IMAGE_FRAME     ( file_path PK, satellite_name+sensor_name FK, location_name FK, capture_time,
cloud_cover, sun_elevation )
OBJECT_TYPE     ( type_name PK, category, object_type_description )
DETECTION_MODEL ( model_name+model_version PK, architecture, trained_on, released_date )
DETECTED_OBJECT ( file_path FK, object_no, type_name FK, model_name+model_version FK, confidence,
bbox..., area_sq_m, attributes )
SPECTRAL_INDEX  ( file_path FK, index_name, mean_value, min_value, max_value )
```

OUR REAL DESIGN This is exactly why we removed `satellite_id` and `resolution` from IMAGE_FRAME, and `basin_id` from DETECTED_OBJECT. Each was reachable through a chain, so storing it again was a transitive dependency.

Step 4: Boyce–Codd Normal Form (BCNF)

BCNF RULE For every non-trivial FD X → Y, **X must be a superkey** (it must identify a whole row). BCNF is stricter than 3NF: 3NF allows a non-key determinant when Y is part of a key; BCNF never does.

Relation	Determinants	Superkeys?
SATELLITE	satellite_name	yes
SENSOR	(satellite_name, sensor_name)	yes

RIVER_BASIN	basin_code; basin_name	yes (both are candidate keys)
LOCATION	location_name	yes
IMAGE_FRAME	file_path; (sensor, location, capture_time)	yes (both are candidate keys)
OBJECT_TYPE / DETECTION_MODEL / DETECTED_OBJECT / SPECTRAL_INDEX	their keys	yes

All relations are in **BCNF**. The decomposition is:

- **Lossless** (lossless-join): every split shares a key, so joining the pieces back gives exactly the original data, with no fake rows.
- **Dependency-preserving**: every FD can be checked inside a single table.

Step 5: Surrogate keys

Finally, the long natural keys (file_path, composite sensor and model keys) were replaced by number IDs (image_id, sensor_id, model_id…), while the natural keys were kept as **UNIQUE**. The tables stay in BCNF, because each natural key is still a candidate key.

The application tables

APP_USER, QUERY_LOG, QUERY_EVIDENCE, ALERT and AUDIT_LOG each have FDs only from their keys (e.g. user_id → all; email → all). They are in BCNF.

3.13 Deliberate denormalization (and why it's justified)

Some stored values could be calculated from other data:

Stored value	Could be derived from	Why we store it	Protection against anomalies
OBJECT_CHANGE: change_pct, time_delta_days, rate_pct_per_month, change_type	the two objects' areas and their images' dates	a **historical measurement**; alerts must show the value that triggered them; faster reports	written **only** by `pkg_eo.compute_changes`; change_type always comes from `fn_classify_change`
ALERT.severity	change_pct	records the severity at the moment of raising	set only by `raise_alerts`
QUERY_EVIDENCE.retrieval_score, rank_position	the scores + the weights	a **snapshot** of one retrieval run; changing weights later must not rewrite old answers	written once per question in one transaction
QUERY_LOG.aggregated_confidence	the evidence rows	shown on every result page	written only by `finalize_query`

Embeddings and attributes_json as JSON (multi-valued)	none	an embedding is always read and written as a whole; attributes differ per object type	CHECK (... IS JSON)

CAREFUL Strictly speaking, **change_pct → change_type** is a dependency between two non-key columns, so OBJECT_CHANGE is not in pure 3NF. Say so openly and give the reasons above. Naming an exception and justifying it scores better than claiming everything is perfect.

3.14 DA1 viva questions

Q: What is the difference between a strong and a weak entity? Give an example from your project.

A strong entity has its own key (SATELLITE). A weak entity can only be identified with its owner's key plus a partial key, and cannot exist without the owner. SENSOR is identified by (satellite, sensor_name), since two satellites both have an "MSI". SPECTRAL_INDEX is identified by (image, index_name).

Q: Why is QUERY_EVIDENCE a separate table?

Questions and objects are many-to-many. A relational table can't store lists, so the M:N becomes a bridge table with both foreign keys. It also has its own attributes (three scores, the final score and the rank), which makes it an associative entity.

Q: Where does the foreign key go in a 1:N relationship, and why?

On the N (many) side, for example image_id inside DETECTED_OBJECT. Each object has exactly one image, so it stores one value. The other way round would need a list.

Q: How did you implement total participation?

As a NOT NULL foreign key. Every image must have a sensor and a location, so those FKs are NOT NULL. Partial participation, like location → basin, is a nullable FK.

Q: Explain 2NF and 3NF with your own example.

2NF removes partial dependencies: with key (file_path, object_no), cloud_cover depends only on file_path, so image data moved to IMAGE_FRAME. 3NF removes transitive dependencies: image → location → country, so LOCATION got its own table.

Q: Why did you remove basin_id from DETECTED_OBJECT?

It was reachable through object → image → location → basin. Storing it again was redundant, and the two could contradict each other.

Q: Is your design fully in BCNF?

Yes for all the core relations. The exceptions are documented derived columns in OBJECT_CHANGE, ALERT, QUERY_EVIDENCE and QUERY_LOG. They are kept as historical snapshots, and only the PL/SQL procedures write them, so they cannot drift.

Q: What is a recursive relationship in your design?

EVOLVED_FROM: a detected object relates to an earlier detected object (the same physical patch). It is implemented as OBJECT_CHANGE with two foreign keys to DETECTED_OBJECT.

Q: What does lossless decomposition mean?

After splitting, a natural join of the pieces gives back exactly the original rows, with no missing and no extra (spurious) tuples. It is guaranteed when the common attribute is a key of at least one piece.

Part 4: DA2 (SQL and PL/SQL Implementation)

What DA2 asked for: implement the DA1 design using SQL and PL/SQL, with table creation, constraints, sample records, SQL queries, PL/SQL programs, a handwritten report and a demonstration.

We did it in **6 scripts**, run in this order:

#	Script	Purpose	Run with
1	schema.sql	15 tables, constraints, 12 indexes, 2 views	F5
2	sample_data.sql	about 490 INSERTs	F5
3	plsql.sql	3 functions, package pkg_eo, 4 triggers	F5
4	run_procedures.sql	runs the PL/SQL and shows results (run once)	F5
5	queries.sql	19 queries + error demos	Ctrl+Enter, one at a time
6	extras.sql	ALTER, GRANT/REVOKE, cursors, exceptions, statement trigger, cascade	F5

4.1 Setting up Oracle

1. Oracle 21c XE was installed (in `D:\Oracle\dbhomeXE`). The project lives in the pluggable database **XEPDB1**.
2. User **JOSHWA** was created in SQL*Plus. SQL Developer connects with host localhost, port 1521 and service name XEPDB1.
3. The first run of schema.sql failed on the views with **ORA-01031: insufficient privileges**. The fix, as the administrator in SQL*Plus:

   ```
   CONNECT / AS SYSDBA
   ALTER SESSION SET CONTAINER = XEPDB1;
   GRANT CREATE VIEW, CREATE PROCEDURE, CREATE TRIGGER, CREATE SEQUENCE TO JOSHWA;
   ```

   A user cannot grant privileges to itself, which is why it had to be done as SYSDBA.

4.2 Oracle data types we used

Type	Stores	Example column
NUMBER(p,s)	numbers with p digits in total, s of them after the decimal point	cloud_cover NUMBER(5,2) → up to 999.99
NUMBER	any number	IDs
VARCHAR2(n)	text up to n bytes	satellite_name VARCHAR2(60)
DATE	date + time to the second	launch_date
TIMESTAMP	date + time with fractions of a second	capture_time
CLOB	very large text	embeddings and attributes stored as JSON text

4.3 schema.sql: creating the tables

A CREATE TABLE explained line by line

```
CREATE TABLE image_frame (
  image_id    NUMBER GENERATED BY DEFAULT ON NULL AS IDENTITY (START WITH 10000),  -- (1)
  sensor_id   NUMBER        NOT NULL,                                               -- (2)
  location_id NUMBER        NOT NULL,
  capture_time TIMESTAMP    NOT NULL,
  cloud_cover NUMBER(5,2)   NOT NULL,
  sun_elevation NUMBER(5,2),                                                        -- (3)
  file_path   VARCHAR2(250) NOT NULL,
  image_embedding CLOB,
  image_created_at TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,                          -- (4)
  CONSTRAINT pk_image_frame PRIMARY KEY (image_id),                                 -- (5)
  CONSTRAINT fk_img_sensor FOREIGN KEY (sensor_id)   REFERENCES sensor(sensor_id),  -- (6)
  CONSTRAINT fk_img_loc    FOREIGN KEY (location_id) REFERENCES location(location_id),
  CONSTRAINT uq_img_capture UNIQUE (sensor_id, location_id, capture_time),          -- (7)
  CONSTRAINT uq_img_path    UNIQUE (file_path),
  CONSTRAINT ck_img_cloud CHECK (cloud_cover BETWEEN 0 AND 100),                    -- (8)
  CONSTRAINT ck_img_sun   CHECK (sun_elevation BETWEEN 0 AND 90),
  CONSTRAINT ck_img_emb   CHECK (image_embedding IS JSON)                           -- (9)
);
```

1. **Identity column**: Oracle numbers new rows automatically. "BY DEFAULT ON NULL" means you may supply a value (the sample data supplies 1–9999), and if you don't, Oracle generates one starting at 10000. That is why the demo detection got id 10000.
2. **NOT NULL**: required. This implements total participation (every image must have a sensor).
3. No NOT NULL: optional.
4. **DEFAULT**: filled in automatically with the current time.
5. **Named constraints** (pk_..., fk_..., ck_..., uq_...): error messages show the name, so you know exactly which rule failed. The app also uses these names to show friendly messages.
6. **Foreign key**: the sensor must exist.
7. **Composite UNIQUE**: a sensor can't photograph the same place twice at the same instant (a candidate key).
8. **CHECK range**.
9. **CHECK IS JSON**: the text must be valid JSON.

Constraint inventory

Type	Count	Examples
PRIMARY KEY	15	one per table
FOREIGN KEY	17	fk_obj_image, fk_loc_basin, fk_chg_prev, fk_chg_curr
UNIQUE	17	uq_satellite_name, uq_sensor(satellite_id, sensor_name), uq_si(image_id, index_name), uq_alert_change
CHECK: range	many	confidence 0–1; latitude −90…90; area > 0; rank > 0; days > 0
CHECK: list of values	10	orbit_type, sensor_type, region_type, change_type, severity, alert_status, user_role, operation

CHECK: multi-column	several	bbox min < max; min ≤ mean ≤ max; prev ≠ curr; resolved_at only when RESOLVED
CHECK: format	5+1	IS JSON on JSON columns; email LIKE '%_@_%._%'
DEFAULT	several	sat_status 'ACTIVE', alert_status 'OPEN', timestamps

What happens when a parent row is deleted (ON DELETE)

Option	Effect	Used for	Reason
CASCADE	children are deleted too	sensor→satellite, object→image, index→image, change/evidence→object, alert→change	the child is meaningless without the parent
SET NULL	children stay; the link is emptied	location→basin, query→user, alert→assigned user	the link is optional
(none) = restrict	the delete is refused (ORA-02292)	image→sensor	don't silently lose image history

Indexes

CONCEPT An **index** is like the index at the back of a book: a sorted structure that lets the database find rows **without scanning the whole table**. Oracle automatically creates an index for every PRIMARY KEY and UNIQUE constraint. We added **12 indexes on foreign-key columns** (e.g. detected_object.image_id). This speeds up joins, and avoids Oracle locking the whole child table when a parent row is deleted.

The data-dictionary count later showed **46 indexes**: our 12, plus the automatic ones for PKs, UNIQUE constraints and LOB (CLOB) columns.

Views

CONCEPT A **view** is a saved SELECT query that you can use like a table. It stores no data of its own. Views simplify repeated joins and can hide complexity.

- **v_image_summary** joins IMAGE_FRAME + SENSOR + SATELLITE + LOCATION + (LEFT JOIN) RIVER_BASIN, giving one row per image with every useful name.
- **v_forest_change** joins OBJECT_CHANGE to both objects, both images, the location and the basin. It gives each change with its location, basin, date, %, rate, both confidences and both cloud covers.

schema.sql starts with a PL/SQL block that **drops the old tables** if they exist, so the script can be re-run safely.

4.4 sample_data.sql: inserting the data

```
INSERT INTO satellite (satellite_id, satellite_name, operator_name, launch_date, orbit_type,
sat_status, revisit_days)
VALUES (1, 'Sentinel-2A', 'ESA/Copernicus', DATE '2015-06-23', 'SUN-SYNCHRONOUS', 'ACTIVE', 10);
```

- Inserts must follow **parent-before-child order** (satellites before sensors before images before objects); otherwise the FK check fails.
- `DATE '2015-06-23'` and `TIMESTAMP '2026-09-29 05:30:00'` are date literals.
- The script ends with **COMMIT**, which saves the inserts permanently.
- OBJECT_CHANGE, ALERT, AUDIT_LOG and the answer confidences are **not inserted**. The PL/SQL creates them.

4.5 Transactions and ACID

CONCEPT A **transaction** is a group of changes that succeed or fail *together*. The ACID properties:

- **Atomicity**: all or nothing.
- **Consistency**: the database moves from one valid state to another, with every rule satisfied.
- **Isolation**: users running at the same time don't see each other's unfinished changes.
- **Durability**: once committed, the changes survive crashes.

Command	Meaning
COMMIT	make the changes permanent and visible to others
ROLLBACK	undo every change since the last commit
SAVEPOINT name	place a bookmark inside a transaction
ROLLBACK TO name	undo only back to the bookmark

CAREFUL **DDL commits automatically** (CREATE, ALTER, DROP, GRANT, COMMENT). That's why the procedure and trigger created in extras.sql stayed even after a ROLLBACK. Also, when you close SQL Developer with pending changes, it asks "Commit or Rollback?". If you're unsure, choose Rollback.

Locks: when one session updates rows and hasn't committed, another session that wants the same rows **waits**. That's what happened when extras.sql seemed "stuck": another connection was holding locks.

4.6 SQL query concepts (with our queries)

Joins

CONCEPT A **JOIN** combines rows from two tables using a matching condition (usually FK = PK). **INNER JOIN** keeps only rows that match on both sides. **LEFT (OUTER) JOIN** keeps all rows from the left table, and fills NULLs where the right side has no match.

Q2: ANTI-JOIN "Locations not in any basin":

```
SELECT l.location_name, l.country, l.region_type
FROM location l LEFT JOIN river_basin b ON b.basin_id = l.basin_id
WHERE b.basin_id IS NULL;
```

The LEFT JOIN keeps every location; WHERE ... IS NULL keeps those without a basin. Result: Chennai Coast, Rotterdam Port, Dubai Marina.

Q4 subtlety: it lists 4 satellites, not 5. Sentinel-1A has no images, and the INNER JOIN drops it. E3 in extras uses a LEFT JOIN and shows Sentinel-1A with 0 images. A good viva example of the difference between the two joins.

Aggregates, GROUP BY and HAVING

CONCEPT **Aggregate functions** (COUNT, SUM, AVG, MIN, MAX, CORR) turn many rows into one value. **GROUP BY** makes one result row per group. **WHERE** filters rows *before* grouping; **HAVING** filters groups *after* grouping.

```
-- Q5: detections per type, only types with more than 5
SELECT t.type_name, t.category, COUNT(*) AS detections, ROUND(AVG(o.confidence_score),3) AS
avg_conf
FROM detected_object o JOIN object_type t ON t.object_type_id = o.object_type_id
GROUP BY t.type_name, t.category
HAVING COUNT(*) > 5
ORDER BY detections DESC;
```

Result: Building 56, Tree Coverage 45, Water Body 40, Vessel 35, Road 25, Cropland 25, Bare Soil 20. Wildfire Scar (2) was removed by HAVING.

Subqueries

CONCEPT A **subquery** is a query inside another query. **Non-correlated**: runs once, independently (Q7: "cloudier than the overall average"). **Correlated**: refers to the outer row, so it conceptually runs once per outer row (Q8: "the latest tree area *for this location*").

```
-- Q7 (non-correlated)
SELECT image_id, location_name, cloud_cover FROM v_image_summary
WHERE cloud_cover > (SELECT AVG(cloud_cover) FROM image_frame);

-- Q8 (correlated: i.location_id comes from the outer query)
... WHERE i.capture_time = (SELECT MAX(i2.capture_time) FROM image_frame i2 ...
                            WHERE i2.location_id = i.location_id ...)
```

Set operators

UNION (combine, removing duplicates), **UNION ALL** (keep duplicates), **INTERSECT** (rows in both), **MINUS** (rows in the first but not the second). **Q9** uses MINUS: "locations with deforestation" MINUS "locations with an alert" = **Manaus, Bhadrachalam**. Their evidence was too cloudy or uncertain, so no alert was raised.

The flagship query: Q10

```
SELECT fc.location_name, fc.basin_name, fc.curr_time, fc.change_pct, fc.rate_pct_per_month,
       ROUND(fn_haversine_km(l.latitude, l.longitude, b.river_ref_lat, b.river_ref_lon)) AS
km_to_river
FROM v_forest_change fc
JOIN location l    ON l.location_id = fc.location_id
JOIN river_basin b ON b.basin_id = l.basin_id
WHERE fc.change_type = 'DEFORESTATION'
  AND fc.curr_time >= TIMESTAMP '2026-07-01 00:00:00'
  AND fc.curr_time <  TIMESTAMP '2026-10-01 00:00:00'
  AND fn_haversine_km(l.latitude, l.longitude, b.river_ref_lat, b.river_ref_lon) <= 300
ORDER BY fc.rate_pct_per_month DESC;
```

It reads: deforestation changes, observed in Q3, at locations within 300 km of their basin's river, fastest first. A PL/SQL function is used inside SQL. The **result is 10 rows**: Bhadrachalam 29-Sep (17.76%, 16.15%/month, 123 km), Manaus 29-Sep (17.77%, 65 km), Kisangani (3 rows, 38 km), Tonle Sap (3 rows, 174 km)…

The `< '2026-10-01'` form (instead of `≤ '2026-09-30'`) safely includes everything on 30 Sep, whatever the time of day.

Window (analytic) functions

CONCEPT A **window function** computes a value over a set of related rows **without collapsing them** (unlike GROUP BY). Syntax: `FUNCTION() OVER (PARTITION BY … ORDER BY …)`. PARTITION BY = "separately for each group"; ORDER BY = "in this order".

```
-- Q12
SUM(change_pct) OVER (PARTITION BY location_id ORDER BY curr_time) AS cumulative_loss_pct,
RANK()          OVER (PARTITION BY curr_time ORDER BY change_pct DESC) AS rank_in_period
```

A running total of loss per location, and a ranking of locations within each date. **LAG()**, used in compute_changes, gets the *previous* row's value in the window. That's how each tree patch is paired with its predecessor.

Hierarchical query (CONNECT BY): Q13

```
SELECT LEVEL AS step,
       SYS_CONNECT_BY_PATH(TO_CHAR(oc.prev_object_id), ' -> ') || ' -> ' || oc.curr_object_id AS
chain
FROM object_change oc
START WITH oc.prev_object_id NOT IN (SELECT curr_object_id FROM object_change)
CONNECT BY PRIOR oc.curr_object_id = oc.prev_object_id;
```

START WITH picks the first links (patches that have no predecessor). **CONNECT BY PRIOR** follows each link to the next one. **LEVEL** is the depth, and **SYS_CONNECT_BY_PATH** builds the path text, e.g. `-> 1 -> 3 -> 5 -> 7 -> 9` (Manaus). 36 rows (9 chains × 4 steps).

Statistics, JSON and CASE

- **Q14: CORR** (correlation, from −1 to +1) between NDVI and tree area = **0.442**. Positive, so the AI detections and the spectral physics agree. It is moderate because

each location starts with a different forest size.

- **Q15: JSON_VALUE** reads fields out of JSON text: vessels by class (cargo 11, avg 127.3 m; tanker 9, 160.7 m; passenger 8; fishing 6).
- **Q16: CASE** turns resolution into classes (High ≤10 m: 30 images; Medium ≤30 m: 30 images).
- **Q11** uses **LEAST/GREATEST** to compute a trust score per event: min(confidences) × (1 − max(cloud)/100). The best is Kisangani 29-Sep at 0.584; the worst Manaus 27-Aug at 0.306.
- **Q17/Q18**: multi-table joins for the evidence list and the alerts dashboard (sorted with `ORDER BY CASE severity …`).
- **Q19**: SAVEPOINT → UPDATE (5 rows) → ROLLBACK TO, so the data is unchanged.
- **Q20**: deliberate errors: ORA-02290 (cloud = 140), ORA-02291 (image 99999 doesn't exist), ORA-02292 (deleting a satellite whose sensor has images).

4.7 PL/SQL basics

Block structure

```
DECLARE                 -- optional: variables, cursors, exceptions
  v_count NUMBER := 0;
BEGIN                   -- required: the statements
  SELECT COUNT(*) INTO v_count FROM satellite;
  DBMS_OUTPUT.PUT_LINE('Satellites: ' || v_count);
EXCEPTION               -- optional: error handling
  WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

- A block with no name is an **anonymous block**. Named blocks are procedures, functions, packages and triggers.
- The **/** on its own line tells SQL Developer/SQL*Plus to run the block.
- **DBMS_OUTPUT.PUT_LINE** prints text. **SET SERVEROUTPUT ON** makes the text appear (it showed in your Script Output).
- **SELECT … INTO** stores a query result in a variable. It must return exactly one row, or it raises NO_DATA_FOUND or TOO_MANY_ROWS.
- **||** joins text together.

Variables and anchored types

- `v_cloud image_frame.cloud_cover%TYPE;` declares a variable with the same type as the column. If the column changes, the code still fits.
- `r c_sat%ROWTYPE;` declares a record with the same fields as a cursor's (or table's) row.
- `c_limit CONSTANT NUMBER := 40;` declares a value that can't change.

Control structures

IF … ELSIF … ELSE … END IF; **CASE**; loops: **LOOP … EXIT WHEN … END LOOP**, **FOR i IN 1..10 LOOP**, and the **cursor FOR loop** `FOR r IN (SELECT …) LOOP`.

Cursors

CONCEPT A **cursor** is a pointer to the result rows of a query, letting PL/SQL process them one at a time. **Implicit cursor**: Oracle creates it for every DML or SELECT INTO; check `SQL%ROWCOUNT` (rows affected). **Explicit cursor**: you declare it with a name. **Parameterised cursor**: takes inputs, like a function. **Cursor attributes**: %ISOPEN (is it open?), %FOUND / %NOTFOUND (did the last FETCH get a row?), %ROWCOUNT (rows fetched so far).

```
-- E3: the long form (OPEN / FETCH / CLOSE)
DECLARE
  CURSOR c_sat IS SELECT ... ;
  r c_sat%ROWTYPE;
BEGIN
  OPEN c_sat;
  LOOP
    FETCH c_sat INTO r;
    EXIT WHEN c_sat%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(r.satellite_name || ' images=' || r.images);
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Rows fetched = ' || c_sat%ROWCOUNT);   -- 5
  CLOSE c_sat;
END;
```

A **cursor FOR loop** (`FOR r IN c_trees LOOP`) does OPEN, FETCH and CLOSE automatically. It's the shorter form.

FOR UPDATE / WHERE CURRENT OF (E4): `FOR UPDATE` locks the selected rows; `UPDATE … WHERE CURRENT OF c_alert` updates the row the cursor is on. It must be a single-table cursor; our first version used a join and failed with ORA-01410 (see 4.11).

Exceptions (error handling)

Kind	Example	Where we used it
Predefined	NO_DATA_FOUND, TOO_MANY_ROWS, DUP_VAL_ON_INDEX, VALUE_ERROR, ZERO_DIVIDE, OTHERS	add_detection catches NO_DATA_FOUND (unknown type), DUP_VAL_ON_INDEX and VALUE_ERROR
User-defined	`e_too_cloudy EXCEPTION;` … `RAISE e_too_cloudy;`	E5: image 18 (53.3% cloud) rejected
Application error	`RAISE_APPLICATION_ERROR(-20020, 'Unknown object type: ...')`	custom numbered errors -20001…-20040 that reach the caller (the app shows them)
Named with a number	`PRAGMA EXCEPTION_INIT(e_no_evidence, -20001)`	in the pkg_eo spec

SQLERRM gives the current error message, and **SQLCODE** its number. Custom error numbers must be between −20000 and −20999.

Procedures, functions and parameters

	Procedure	Function

Returns	nothing directly (it can use OUT parameters)	exactly one value (RETURN)
Called	as a statement: `EXEC pkg_eo.compute_changes;`	inside expressions and SQL: `WHERE fn_haversine_km(...) <= 300`
Used for	actions (insert, update, report)	calculations

Parameter modes: **IN** (input, the default), **OUT** (output to the caller), **IN OUT** (both). **DEFAULT** values make parameters optional (raise_alerts). **DETERMINISTIC** marks a function that always returns the same output for the same input (fn_haversine_km).

Packages

CONCEPT A **package** groups related procedures, functions, variables and exceptions. It has two parts: the **specification** (the public interface: what can be called) and the **body** (the code). Benefits: organisation, encapsulation (hide internal details), shared definitions, and it is loaded into memory in one go. It is like a class with only public methods.

Triggers

CONCEPT A **trigger** is PL/SQL that runs **automatically** when an event happens on a table. **Timing**: BEFORE (can check or modify the new values) or AFTER (react, e.g. write an audit row). **Event**: INSERT, UPDATE (optionally *OF specific columns*), DELETE. **Level**: **row-level** (`FOR EACH ROW`, fires once per affected row) or **statement-level** (fires once per statement, however many rows). **:OLD / :NEW**: the row before and after the change (row-level only). **INSERTING / UPDATING / DELETING**: booleans telling you which event fired.

Triggers are used for rules that CHECK constraints can't express, such as rules that involve other rows, other tables or the current time.

4.8 plsql.sql: our programs

Functions

fn_haversine_km(lat1, lon1, lat2, lon2) computes the great-circle distance on a sphere of radius 6371 km:

$$a = \sin^2(\Delta\varphi/2) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2(\Delta\lambda/2) \qquad d = 2R \cdot \operatorname{atan2}(\sqrt{a}, \sqrt{(1-a)})$$

(φ = latitude, λ = longitude, both in radians.) We need it because Earth is round, so plain Pythagoras on degrees is wrong. Example: Kisangani (0.80, 25.00) to the Congo reference point (0.5167, 25.1833) is about **38 km**.

fn_classify_change(pct): ≥10 DEFORESTATION; ≤−10 REGROWTH; ≥3 GRADUAL_LOSS; otherwise STABLE.

fn_forest_loss_pct(location, from, to) takes the first and last tree area in the window using `MAX(area) KEEP (DENSE_RANK FIRST/LAST ORDER BY capture_time)`, then computes (first − last)/first × 100.

Package pkg_eo

```
CREATE OR REPLACE PACKAGE pkg_eo AS
  e_no_evidence EXCEPTION;  PRAGMA EXCEPTION_INIT(e_no_evidence, -20001);
  PROCEDURE compute_changes;
  PROCEDURE raise_alerts (p_min_loss NUMBER DEFAULT 10, p_min_conf NUMBER DEFAULT 0.7,
p_max_cloud NUMBER DEFAULT 40);
  FUNCTION  fn_query_confidence (p_query_id NUMBER) RETURN NUMBER;
  PROCEDURE finalize_query (p_query_id NUMBER);
  PROCEDURE add_detection (p_image_id NUMBER, p_type_name VARCHAR2, p_model_id NUMBER, p_conf
NUMBER,
                           p_lat NUMBER, p_lon NUMBER, p_area NUMBER, p_new_id OUT NUMBER);
  PROCEDURE basin_report (p_basin_code VARCHAR2);
END pkg_eo;
```

Member	What it does, step by step
compute_changes	An explicit cursor selects every Tree Coverage patch with its image time, and uses **LAG()** partitioned by location to fetch the previous patch's id, area and time. For each row with a predecessor: change % = (prev − curr)/prev × 100, days = the date difference, rate = %/days × 30, type = fn_classify_change. **MERGE** updates the link if it exists, otherwise inserts it, so re-running is safe. Then COMMIT and print the count (36).
raise_alerts	Loops over changes with loss ≥ p_min_loss, **both** detections' confidence ≥ p_min_conf, **both** images' cloud ≤ p_max_cloud, and **NOT EXISTS** an alert already. Severity: ≥25 CRITICAL, ≥15 HIGH, else MEDIUM. Inserts the alert as OPEN.
fn_query_confidence	Returns Σ(retrieval_score × confidence × (1 − cloud/100)) / Σ retrieval_score over the question's evidence. `NULLIF(x,0)` avoids division by zero.
finalize_query	If the question has no evidence, raises −20001. Otherwise computes the confidence, builds a sentence ("Answer based on 5 evidence objects. Overall confidence 61.2% (moderate...)") and UPDATEs QUERY_LOG.
add_detection	SAVEPOINT → look up the type id (NO_DATA_FOUND → −20020 "Unknown object type") → compute a small bbox from the area → INSERT ... **RETURNING object_id INTO p_new_id** → COMMIT. On error: ROLLBACK TO the savepoint, so nothing half-done stays.
basin_report	A **parameterised cursor** c_loc(cp_code) lists the basin's locations. For each, it calls fn_forest_loss_pct and prints a padded line. A BOOLEAN flag reports "No locations found" for a wrong code.

Triggers

Trigger	Timing / level	Purpose
trg_obj_audit	AFTER INSERT/UPDATE/DELETE on DETECTED_OBJECT, row	writes old/new confidence and area to AUDIT_LOG, using INSERTING/UPDATING and :OLD/:NEW
trg_img_validate	BEFORE INSERT/UPDATE OF capture_time, row	rejects future capture times (−20010). A CHECK can't use SYSTIMESTAMP

trg_alert_resolve	BEFORE UPDATE OF alert_status, row	sets :NEW.resolved_at := SYSTIMESTAMP when the status becomes RESOLVED; clears it otherwise
trg_change_validate	BEFORE INSERT/UPDATE on OBJECT_CHANGE, row	both objects at the same location (−20030); previous older than current (−20031)
trg_alert_stmt_audit (extras)	AFTER INSERT/UPDATE/DELETE on ALERT, **statement**	one audit row per statement

4.9 run_procedures.sql: the proof (your actual output)

```
compute_changes: 36 lineage links created/updated.
raise_alerts: 4 new alerts.
finalize_query(1): Answer based on 5 evidence objects. Overall confidence 61.2% (moderate ...)
finalize_query(2): ... 66.6% ...   finalize_query(3): ... 61.8% ...   finalize_query(4): ...
60.1% ...
=== Forest loss report, basin MKG ===   Kratie Riverbank 7.74 % loss   Tonle Sap Fringe 38.49 %
loss
=== Forest loss report, basin AMZ ===   Manaus Forest Edge 38.72 % loss  Rondonia Frontier 8.13 %
loss
Inserted detection id = 10000
Expected error caught: ORA-20020: Unknown object type: Spaceship
Expected error caught: ORA-20010: capture_time cannot be in the future ...
```

Output	What it proves
36 links	9 forest locations × 4 consecutive image pairs; the lineage builder works
4 alerts (of 10 deforestation events)	trust filtering works: 6 events lacked confident, clear-sky evidence
0.6121 / 0.6655 / 0.618 / 0.6006	the confidence formula works, and it matched the independent Python calculation to 4 decimals
basin reports	the parameterised cursor + function; rapid ≈ 38% vs slow ≈ 8% over 6 months
id 10000 inserted	add_detection + the OUT parameter + identity start; the count goes 247 → **248**
ORA-20020	package validation + savepoint rollback
ORA-20010	the future-date trigger works
alert 10000 RESOLVED with resolved_at set	trg_alert_resolve works
2 audit rows (INSERT 10000; UPDATE object 1 conf .837 → .99)	trg_obj_audit works

CAREFUL Run run_procedures.sql **once**. Each run inserts another demo detection (249, 250…). To clean up: `DELETE FROM detected_object WHERE object_id > 10000; COMMIT;`

4.10 extras.sql: extra coverage (your results)

Part	Concept	Result

E1	DDL: ALTER TABLE ADD / MODIFY / ADD CONSTRAINT / RENAME COLUMN / DROP COLUMN; DISABLE/ENABLE CONSTRAINT; COMMENT ON	masses listed; CK_IMG_CLOUD DISABLED → ENABLED; comment shown
E2	DCL: GRANT / REVOKE, user_tab_privs_made	3 privileges → 0
E3	explicit cursor + attributes	5 satellites (Sentinel-1A: 0 images); %ROWCOUNT = 5
E4	FOR UPDATE + WHERE CURRENT OF	alert 10002 (13.91%) escalated → rolled back
E5	user-defined exception, %TYPE, CONSTANT	image 23 (3.7%) accepted; image 18 (53.3%) rejected
E6	standalone procedure, IN/OUT parameters	Sentinel-2A: 15 images, 28% cloud, 61 objects; "Hubble" → ORA-20040
E7	statement-level trigger	3 rows updated, but only 1 audit row
E8	ON DELETE CASCADE inside a SAVEPOINT	image 60's objects 6 → 0 → 6 after rollback
E9	data dictionary USER_OBJECTS	3 functions, 46 indexes, 1 package + body, 1 procedure, 15 tables, 5 triggers, 2 views

The **data dictionary** is Oracle's own set of tables describing your objects: USER_TABLES, USER_CONSTRAINTS, USER_OBJECTS, USER_TAB_PRIVS_MADE, USER_TAB_COMMENTS…

4.11 Problems we hit and fixed (good to mention)

Error	Cause	Fix
ORA-01031	JOSHWA lacked the CREATE VIEW privilege	GRANT as SYSDBA
ORA-04098 (invalid trigger)	`CASE WHEN INSERTING` was used inside a SQL INSERT; INSERTING is PL/SQL-only, so the trigger compiled invalid and every statement touching ALERT failed (even a DELETE on objects, via the cascade)	evaluate INSERTING/UPDATING in IF statements into a variable
ORA-01410 (invalid ROWID)	WHERE CURRENT OF on a cursor that joined two tables	cursor on ALERT only; look up the change % inside the loop
Script "stuck"	another session held row locks	commit or close the other session; cancel the task
Count 250 instead of 248	run_procedures.sql run more than once	delete the extra demo rows (> 10000)

4.12 DA2 viva questions

Q: Procedure vs function?

A function returns one value and can be used inside SQL (fn_haversine_km in Q10's WHERE). A procedure performs an action and returns values only through OUT parameters (add_detection returns the new id).

Q: Why use a package?

It groups related code (all the EO logic) with a public spec and a private body, shares definitions such as the e_no_evidence exception, and compiles and loads as one unit.

Q: Row-level vs statement-level trigger?

Row-level fires once per affected row and can use :OLD/:NEW; statement-level fires once per statement. In E7, updating 3 alerts added only 1 audit row.

Q: Why a trigger for "no future dates" instead of a CHECK?

CHECK constraints can't use SYSDATE or SYSTIMESTAMP, because the result would change over time. A trigger can compare with the current time.

Q: What does MERGE do?

Insert-or-update in one statement: if a matching row exists, update it; otherwise insert. It makes compute_changes safe to re-run.

Q: CASCADE vs SET NULL — where did you use each?

CASCADE where the child has no meaning without the parent (objects of an image). SET NULL where the link is optional (location → basin).

Q: Correlated vs non-correlated subquery?

Non-correlated runs once on its own (Q7, the average cloud). Correlated refers to the outer row and runs per row (Q8, the latest image for *this* location).

Q: Why only 4 alerts when there are 10 deforestation events?

raise_alerts also requires confidence ≥ 0.7 and cloud ≤ 40% on both images. Six events had cloudy or low-confidence evidence, so no false alarms were raised. Q9 lists two such locations (Manaus, Bhadrachalam).

Q: What is SAVEPOINT used for in add_detection?

If anything fails after the savepoint, ROLLBACK TO undoes only this procedure's work and leaves the caller's other changes intact.

Q: WHERE vs HAVING?

WHERE filters rows before grouping; HAVING filters groups after aggregation (Q5: HAVING COUNT(*) > 5).

Q: What is %TYPE and why use it?

It declares a variable with the same type as a column, so the code stays correct if the column's type changes.

Part 5: The Novelty (What Makes This Project Special)

5.1 Why novelty was needed

Combining "satellite data + knowledge graph" or "GraphRAG for geospatial data" already exists in published work (for example a 2025 ISWC paper on virtual knowledge graphs over EO data, and GeoGraphRAG). So just building such a system is not new. We looked for what those systems do **not** handle well, and added three ideas. All three are implemented **inside the database**, not just described.

#	Idea	One-line summary
1	Temporal lineage (EVOLVED_FROM)	link the same physical object across time, so change is measured and explainable
2	Tunable hybrid retrieval	rank evidence by meaning + place + time, with adjustable weights α, β, γ
3	Confidence propagation	carry detection confidence and cloud cover into a trust score for every answer and alert

5.2 Novelty 1: Temporal lineage

The problem

Most systems treat each detection as an independent record. The forest patch in July and the same patch in September are unrelated rows. To decide whether a forest is shrinking, they rely on a **fuzzy similarity guess** ("these two look alike, so they're probably the same place"), which is hard to explain or verify.

Our solution

Each new observation is **explicitly linked to its previous observation** of the same place, and the link stores the change.

```
Manaus tree patch:   object 1 (10 Apr) -> 3 (18 Jun) -> 5 (22 Jul) -> 7 (27 Aug) -> 9 (29 Sep)
Kisangani tree patch: 33 -> 35 -> 37 -> 39 -> 41
```

Each link (a row of OBJECT_CHANGE) stores:

- **change_pct** = (previous area − current area) / previous area × 100
- **time_delta_days** = the days between the two images
- **rate_pct_per_month** = change_pct / days × 30. This is what makes "**rapid**" a number.
- **change_type** = DEFORESTATION (≥10%), GRADUAL_LOSS (≥3%), STABLE, REGROWTH (≤−10%)

WORKED EXAMPLE Manaus, 27 Aug → 29 Sep: the tree area fell by **17.77%** in **33 days**. Rate = 17.77 / 33 × 30 = **16.15% per month**, so DEFORESTATION.

Benefits

- **Explainable**: you can show the exact chain of evidence.
- **Measurable**: "rapid" is a rate, not an opinion.
- **Queryable**: the whole history comes back in one query (CONNECT BY in SQL, `-[:EVOLVED_FROM*]->` in Neo4j).

Where it is implemented

Layer	Implementation
DA1	OBJECT_CHANGE associative entity (a recursive relationship on DETECTED_OBJECT)
DA2	pkg_eo.compute_changes (LAG + MERGE, 36 links); trg_change_validate (same location, time order); Q12 and Q13
DA3	a real edge `(:DetectedObject)-[:EVOLVED_FROM]->(:DetectedObject)`; queries G3–G5
App	location page: the chain, the change table, and the tree-area + NDVI chart

5.3 Novelty 2: Tunable hybrid retrieval score

The problem

Standard semantic search (RAG) ranks results by **meaning only**. Our question also has a **place** and a **time**. Ranking by meaning alone can return a perfect-looking deforested patch that is far from any river, or from the wrong season.

Our solution

```
retrieval_score = α × semantic + β × spatial + γ × temporal
```

Component	Question it answers	Formula
semantic	does it look like the concept?	max(0, cosine(object embedding, query embedding))
spatial	is it near a river?	$e^{-km\ /\ 200}$, using the distance to the nearest basin river point
temporal	is it inside the time window?	1.0 inside, 0.3 outside

Default weights: **α = 0.5, β = 0.3, γ = 0.2**. They add up to 1, so the score stays between 0 and 1. The weights are **stored with each question** (QUERY_LOG.retrieval_weights_json), so every answer records how it was ranked.

WHY $E^{-KM/200}$? An **exponential decay** gives 1.0 at the river and falls smoothly with distance, never below 0:

Distance	0 km	38 km	65 km	100 km	174 km	200 km	300 km
Spatial score	1.00	0.83	0.72	0.61	0.42	0.37	0.22

200 km is the "scale": at 200 km the score is $e^{-1} \approx 0.37$.

WORKED EXAMPLE The top evidence for question 1 was the **Kisangani tree patch on 29 Sep**:

- semantic ≈ 1.00 (it looks exactly like the deforestation concept)
- spatial = $e^{-38/200}$ = 0.827 (38 km from the Congo)
- temporal = 1.0 (29 Sep is in Q3)

Score = 0.5 × 1.00 + 0.3 × 0.827 + 0.2 × 1.0 = 0.500 + 0.248 + 0.200 = **0.948** (stored as 0.9484).

Benefits

- **Tunable**: "near the port" questions can raise β; "recent changes" questions can raise γ.
- **Transparent**: the app shows each evidence row's score mix as a coloured bar.
- **Experimentable**: change the weights, measure answer quality, and report which works best (a natural research experiment).

Where the query vector comes from (in the app)

The app has no language model, so it cannot turn free text into a vector. Instead the user picks a **concept** (e.g. "rapid forest loss"), and its vector is the **centroid** (average, then normalised) of **real labelled examples in the database**. For deforestation, that is the embeddings of tree patches already classified as DEFORESTATION. This makes the concept data-driven.

Where it is implemented

Layer	Implementation
DA1	QUERY_EVIDENCE stores the 3 scores + the final score + rank; QUERY_LOG stores the weights
DA2	20 evidence rows; Q17 shows the top-3 per question
DA3	G7 computes it live in Neo4j (vector index + point.distance + datetime)
App	the Ask (GraphRAG) page: set α, β, γ; the evidence is ranked, saved and displayed

5.4 Novelty 3: Confidence propagation and trust-aware alerts

The problem

AI detections are uncertain (confidence 0.58 vs 0.91), and cloudy images are less reliable. Most systems discard this and give a flat answer.

Our solution: the answer confidence

```
answer_confidence = Σ [ retrieval_score × detection_confidence × (1 −
                cloud_cover/100) ] / Σ retrieval_score
```

In words: a **weighted average of how trustworthy each piece of evidence is**. A piece of evidence is trustworthy when the AI was confident and the image was clear, and better-ranked evidence counts more.

WORKED EXAMPLE (ILLUSTRATIVE NUMBERS) Two evidence rows:

Row	retrieval	confidence	cloud	trust = conf × (1 − cloud/100)	retrieval × trust
A	0.95	0.80	10%	0.80 × 0.90 = 0.72	0.684
B	0.90	0.70	40%	0.70 × 0.60 = 0.42	0.378

Confidence = (0.684 + 0.378) / (0.95 + 0.90) = 1.062 / 1.85 = **0.574 → 57.4% (moderate)**.

Reading the score: ≥ 75% high; 50–75% moderate; < 50% low ("treat with caution"). Our four sample questions scored **61.2%, 66.6%, 61.8%, 60.1%** (moderate), because some evidence came from cloudy images.

The same idea for alerts

pkg_eo.raise_alerts raises an alert only if: loss ≥ 10% **and** both detections have confidence ≥ 0.7 **and** both images have ≤ 40% cloud. Result: **10 deforestation events → 4 alerts**. The other 6 (for example Manaus 27-Aug, whose earlier image had 44.6% cloud and confidence 0.553) were deliberately not alerted. This prevents **false alarms**.

Benefits

- The system says **how sure it is**, which matters before sending rangers or making policy.
- It connects database-level data (confidence 0–1 and cloud 0–100, enforced by CHECK) to answer-level trust.
- Verified: the PL/SQL result matched an independent Python calculation to 4 decimals.

Where it is implemented

Layer	Implementation
DA1	aggregated_confidence (derived); confidence_score and cloud_cover with CHECK ranges
DA2	fn_query_confidence, finalize_query, raise_alerts; Q9 (no-alert events), Q11 (trust ranking)
DA3	G7 returns answer_confidence; G8 reproduces exactly the 4 trusted alerts
App	the big confidence number on the Ask result page; the "Trust" column in the Finder; the alert thresholds form

5.5 How the three ideas answer the flagship question together

> "*Find images showing rapid deforestation near river basins in Q3*"

1. **Lineage** finds tree patches that shrank quickly between consecutive images (rapid deforestation).
2. **Hybrid scoring** ranks the candidates by "looks like deforestation" × "near a river" × "in Q3".
3. **Confidence propagation** tells you how much to trust the answer, and only trustworthy changes become alerts.

5.6 Honest limitations of the novelty

- **Synthetic embeddings**: 8-number vectors designed so deforested patches cluster. A real system would use satellite-image foundation models (e.g. SatCLIP, 512+ numbers).
- **Matching the same patch** is simple here: one tree patch per location per image. Real imagery needs matching by overlap, shape and similarity, which is a harder research problem.
- **Distance** is to one river reference point per basin, not the actual river line.
- **No language model** writes the final answer; a template sentence is used. Connecting an LLM is future work.

ONE-LINE ANSWER "Our novelty is threefold: we explicitly track the same object over time so change is measurable and explainable; we rank evidence with a tunable mix of meaning, location and time; and we propagate detection confidence and cloud cover into a trust score for every answer and alert."

Part 6: DA3 (Neo4j Graph Database Extension)

What DA3 asked for: extend the same mini-project with one modern database technology (not MongoDB), and show the database schema/model, the implementation, a sample dataset, database operations, technology-specific queries, and a comparison explaining why the chosen database suits the application. We chose **Neo4j** (graph database), running as a local instance **eo** in Neo4j Desktop 2.3, version 2026.10.0, with the Cypher 25 language. No plugins were needed.

6.1 Why a graph database?

Our data is mostly **connections**:

```
Satellite <- Sensor <- Image -> Location -> River basin
                         ^
                Detected object -> (its earlier self) -> (its earlier self) ...
```

In Oracle, each hop is a **JOIN**, computed at query time by matching keys. Q10 needed about 6 joins, and the lineage history needed a special CONNECT BY query. In a graph database, each connection is **stored directly** as a pointer from one node to the next ("index-free adjacency"). Following a link is like walking along a road that is already built, instead of looking up an address in a phone book every time.

Neo4j also offers two features Oracle XE lacks:

- **Spatial points**: a built-in point type, `point.distance()`, `point.withinBBox()` and point indexes.
- **Vector indexes**: native similarity search over embeddings, the semantic half of GraphRAG.

6.2 The property-graph model

Term	Meaning	Our example
Node	an entity instance (like a row)	one image
Label	the node's type (like a table name)	:ImageFrame, :Satellite
Relationship	a named, **directed** link between two nodes (like an FK, but stored)	(image)-[:CAPTURED_BY]->(satellite)
Property	a key–value pair on a node *or* a relationship	cloud_cover: 31.4; change_pct: 17.77 (on EVOLVED_FROM)

A graph has no tables, no foreign-key columns and no joins. The relationship **is** the connection. The model is also **schema-flexible**: you can add a property or a label to some nodes at any time, without an ALTER TABLE.

6.3 Cypher: the query language

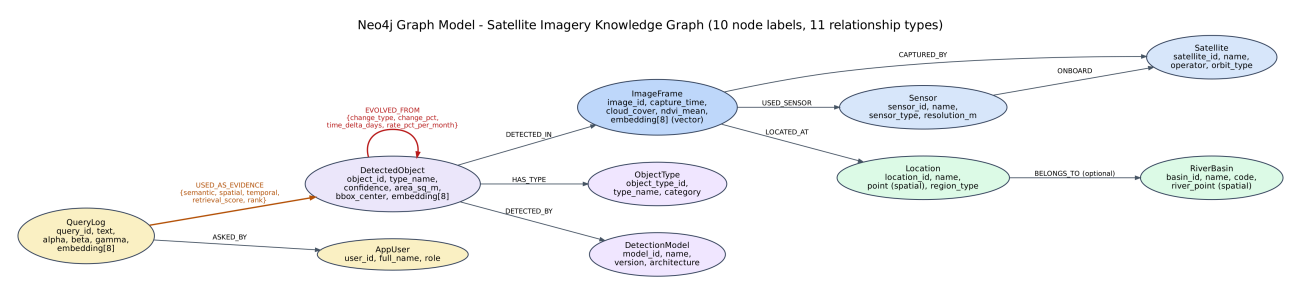
Cypher looks like ASCII art: `( )` is a node, `-[ ]->` is a relationship with a direction.

```
MATCH (i:ImageFrame)-[:LOCATED_AT]->(l:Location {name: 'Kratie Riverbank'})
RETURN i.image_id, i.cloud_cover
```

"Find images located at Kratie Riverbank and return their id and cloud cover."

Cypher	Like in SQL	Meaning
MATCH	FROM + JOIN	find this pattern
OPTIONAL MATCH	LEFT JOIN	match if present, else null
WHERE	WHERE	filter
RETURN	SELECT	output
WITH	a subquery / pipeline step	pass results to the next part of the query
ORDER BY, LIMIT	ORDER BY, FETCH FIRST	sort, limit
count(), avg(), collect()	COUNT, AVG, (list aggregation)	aggregation (grouping is implicit by the other returned columns)
CREATE	INSERT	always creates
MERGE	MERGE	match if it exists, otherwise create
SET / REMOVE	UPDATE / ALTER	change or remove properties and labels
DELETE / DETACH DELETE	DELETE	remove a node (DETACH also removes its relationships)
-[:R*]->	recursive CTE / CONNECT BY	a path of any length

6.4 The graph model (schema)



10 node labels: Satellite, Sensor, ImageFrame, Location, RiverBasin, ObjectType, DetectionModel, DetectedObject, QueryLog, AppUser.

11 relationship types: ONBOARD (Sensor→Satellite), CAPTURED_BY (Image→Satellite), USED_SENSOR (Image→Sensor), LOCATED_AT (Image→Location), BELONGS_TO (Location→Basin), DETECTED_IN (Object→Image), HAS_TYPE (Object→Type), DETECTED_BY (Object→Model), **EVOLVED_FROM** (newer Object→older Object, with change properties), **USED_AS_EVIDENCE** (Query→Object, with score properties), ASKED_BY (Query→User).

How each SQL table was converted

Relational (Oracle)	Graph (Neo4j)	Reason
10 entity tables	10 node labels	each row becomes a node
foreign-key columns	relationships	the stored link replaces the FK
OBJECT_CHANGE	EVOLVED_FROM relationship with properties	its only job was linking two objects
QUERY_EVIDENCE (M:N bridge)	USED_AS_EVIDENCE relationship with properties	a graph needs no bridge table for M:N
SPECTRAL_INDEX	ndvi_mean, ndwi_mean properties on ImageFrame	small one-per-image data
latitude/longitude	`point({latitude, longitude})`	native distance + spatial index
embedding JSON text	a list of floats + a vector index	native similarity search
ALERT, AUDIT_LOG	not copied	operational records stay in Oracle (the system of record); G8 computes alerts on the fly
type_name only in OBJECT_TYPE	also copied onto DetectedObject	deliberate denormalization: saves a hop on frequent filters

Satellite is reachable both directly (CAPTURED_BY) and via the sensor (USED_SENSOR → ONBOARD). Graphs often add such **shortcut edges** for convenience.

6.5 Implementation

Setup steps

1. Neo4j Desktop → create instance "eo" (version 2026.10.0, user neo4j, a password) → Start.
2. Open folder → **import** → copy the 12 CSV files from `neo4j/import`. LOAD CSV can only read from this folder, for security.
3. Connect → Query, then run the scripts in order: 01 → 02 → 03 → 04 → 05.

01_constraints_indexes.cypher (17 statements)

```
CREATE CONSTRAINT image_pk IF NOT EXISTS FOR (n:ImageFrame) REQUIRE n.image_id IS UNIQUE;
CREATE INDEX image_time_idx IF NOT EXISTS FOR (n:ImageFrame) ON (n.capture_time);
CREATE POINT INDEX location_point_idx IF NOT EXISTS FOR (n:Location) ON (n.point);
CREATE VECTOR INDEX object_embedding_idx IF NOT EXISTS
FOR (n:DetectedObject) ON (n.embedding)
OPTIONS {indexConfig: {`vector.dimensions`: 8, `vector.similarity_function`: 'cosine'}};
```

- **11 uniqueness constraints**, the graph equivalent of primary/unique keys.
- **2 property indexes** (capture_time, type_name) for fast filtering.
- **2 point indexes** (Location.point, RiverBasin.river_point) for fast spatial search.
- **2 vector indexes** (8 dimensions, cosine) on DetectedObject and ImageFrame embeddings.
- **IF NOT EXISTS** makes the script safe to re-run.

02_load.cypher: loading the CSVs

```
LOAD CSV WITH HEADERS FROM 'file:///images.csv' AS row
MERGE (i:ImageFrame {image_id: toInteger(row.image_id)})
SET i.capture_time = datetime(row.capture_time),
    i.cloud_cover  = toFloat(row.cloud_cover),
    i.embedding    = [x IN split(row.embedding, ';') | toFloat(x)]
WITH i, row
MATCH (sat:Satellite {satellite_id: toInteger(row.satellite_id)})
MATCH (loc:Location  {location_id:  toInteger(row.location_id)})
MERGE (i)-[:CAPTURED_BY]->(sat)
MERGE (i)-[:LOCATED_AT]->(loc);
```

- **LOAD CSV WITH HEADERS** reads each line as a map (`row.column_name`). Every value arrives as text.
- Type conversion: `toInteger, toFloat, datetime(), date(), point()`.
- `[x IN split(…, ';') | toFloat(x)]` is a **list comprehension** that turns "0.12;0.55;..." into a list of numbers.
- **MERGE** (not CREATE) means re-running creates no duplicates.
- **WITH** passes the node and row on to the next part, which links the relationships.

Result: 354 nodes. DetectedObject 247, ImageFrame 60, Location 12, ObjectType 8, Sensor 6, Satellite 5, RiverBasin 5, QueryLog 4, AppUser 4, DetectionModel 3. Relationships include EVOLVED_FROM 36 and USED_AS_EVIDENCE 20.

CAREFUL Neo4j has **247** detected objects and Oracle has **248**. The extra Oracle row is the PL/SQL demo insert (id 10000), which is not in the CSVs. Expected.

6.6 The technology-specific queries (03_queries.cypher)

Query	What it shows	Feature
G1	satellite → image → location, with object counts	basic pattern match
G2	object → image → location → basin in one MATCH	multi-hop (SQL needed 4 joins)
G3	flagship: rapid deforestation near rivers in Q3, **10 rows**	path + point.distance + datetime
G4	Kisangani's patch history: area at each step	variable-length path
G5	total loss per location, ranked	variable-length paths + aggregation
G6	"what looks like deforestation?"	vector index search
G7	full GraphRAG: hybrid score + answer confidence	vector + spatial + temporal
G8	trustworthy changes = the 4 Oracle alerts	filters on relationship properties
G9	locations within 1500 km of Phnom Penh	point.distance
G10	locations inside a South-East Asia box	point.withinBBox
G11	patches most similar to the worst one	vector search seeded from a node
G12	shortest path from a patch to its satellite	shortestPath
G13	images, cloud and NDVI per satellite	aggregation
G14	add a new observation into a lineage, then clean up	write query

G3 explained (the flagship)

```
MATCH (b:RiverBasin)<-[:BELONGS_TO]-(l:Location)<-[:LOCATED_AT]-(i:ImageFrame)
      <-[:DETECTED_IN]-(o:DetectedObject)-[e:EVOLVED_FROM]->(p:DetectedObject)
WHERE o.type_name = 'Tree Coverage' AND e.change_type = 'DEFORESTATION'
  AND i.capture_time >= datetime('2026-07-01T00:00:00')
  AND i.capture_time <  datetime('2026-10-01T00:00:00')
  AND point.distance(l.point, b.river_point) <= 300000          // metres
RETURN l.name, b.name, date(i.capture_time), e.change_pct, e.rate_pct_per_month
ORDER BY e.rate_pct_per_month DESC;
```

Read it as a sentence: "a basin, a location in it, an image of the location, a tree patch in the image that evolved from an earlier patch, where the change was deforestation, in Q3, within 300 km of the river". The path pattern replaces 6 joins; point.distance replaces our haversine function (it returns **metres**); the change data sits on the relationship e. It returns the **same 10 rows as SQL Q10**, so the two databases agree.

G4: variable-length paths

```
MATCH path = (latest:DetectedObject)-[:EVOLVED_FROM*]->(first:DetectedObject)
WHERE NOT ()-[:EVOLVED_FROM]->(latest) AND NOT (first)-[:EVOLVED_FROM]->()
...
RETURN [n IN nodes(path) | round(n.area_sq_m / 1000000, 2)] AS area_km2_newest_to_oldest,
       [r IN relationships(path) | r.change_pct] AS loss_pct_per_step
```

`*` means "one or more hops". The two NOT conditions keep only the **complete** chains (latest has nothing newer; first has nothing older). `nodes(path)` and `relationships(path)` pull the lists out of the path.

G6 and G7: vector search and GraphRAG

```
MATCH (q:QueryLog {query_id: 1})
CALL db.index.vector.queryNodes('object_embedding_idx', 40, q.embedding) YIELD node AS o, score
AS semantic
MATCH (o)-[:DETECTED_IN]->(i:ImageFrame)-[:LOCATED_AT]->(l:Location)
CALL (l) { MATCH (b:RiverBasin) RETURN min(point.distance(l.point, b.river_point)) / 1000.0 AS km
}
WITH q, o, i, l, semantic, exp(-km / 200.0) AS spatial,
     CASE WHEN i.capture_time >= datetime('2026-07-01T00:00:00')
           AND i.capture_time <  datetime('2026-10-01T00:00:00') THEN 1.0 ELSE 0.3 END AS
temporal
WITH ..., q.alpha*semantic + q.beta*spatial + q.gamma*temporal AS retrieval_score
ORDER BY retrieval_score DESC LIMIT 5
... RETURN top5, answer_confidence
```

1. **queryNodes** asks the vector index for the 40 objects whose embeddings are most similar to the question's embedding (*semantic*).
2. **CALL (l) { ... }** is a subquery run for each location; it finds the distance to the nearest river (*spatial*). This is Cypher 25 syntax: the older `CALL { WITH l … }` form failed, so we updated it.
3. The CASE gives the *temporal* score; the weights α, β, γ are read from the question node.
4. Keep the top 5, then compute the answer confidence with the same formula as the PL/SQL.

CAREFUL (1) Neo4j reports cosine scaled to **(1 + cos)/2**, so its scores differ slightly from SQL/Python and the confidence is close to, but not exactly, 0.6121. (2) The warning "db.index.vector.queryNodes is deprecated, replaced by SEARCH" is only a notice. The query works, and Cypher 25 adds a newer SEARCH clause for vector search.

6.7 Database operations (04_crud.cypher)

Step	Operation	What it shows
C1	CREATE a satellite, sensor and image with links	inserting connected data in one statement
C2	MATCH p = (i)-[]-(n) RETURN p	read, shown in the graph view
C3	SET cloud_cover and a new property	update
C4	REMOVE a property; add/remove a label	schema-flexible update (no ALTER needed)
C5	MERGE ... ON CREATE SET ... ON MATCH SET	upsert without duplicates
C6	CREATE a duplicate satellite_id	**expected error**: the uniqueness constraint works
C7	images per satellite	aggregation including the new data
C8	plain DELETE (fails), then DETACH DELETE	Neo4j refuses to leave dangling relationships
C9	counts	back to the original dataset

LESSON LEARNED In a multi-statement run, Neo4j stops at the first error and **skips the rest**. The C8 cleanup was skipped after the expected error, so it had to be run on its own. Run "expected error" statements alone.

6.8 Graph views (05_graph_views.cypher)

A table view appears when a query returns plain values. The **Graph** view appears only when a query returns nodes, relationships or paths. 05_graph_views.cypher returns paths for each question: V0 is the schema (`CALL db.schema.visualization()`), V3 the flagship result, V4 a lineage chain, V7 question → evidence → image → location → basin, V8 the trusted alerts, and so on. Set node captions to `name` for readable screenshots.

6.9 The comparison (required)

Need	Oracle SQL (DA2)	Neo4j (DA3)
Integrity rules (CHECK, FK, triggers)	**strong**	limited (uniqueness/existence constraints)
Multi-hop connections	many joins	**one path pattern**
History of unknown length	CONNECT BY	**-[:EVOLVED_FROM*]->**
Many-to-many	a bridge table	**a direct relationship** with properties
"Near a river"	hand-written function, no spatial index (XE)	**native point.distance + point index**

"Looks like deforestation"	not practical	**native vector index**
Changing structure	ALTER TABLE	**flexible**: add properties/labels any time
Aggregation and reporting	**excellent**	good
Transactions (ACID)	yes	yes

CONCLUSION TO SAY "Our application is about connections, location and meaning, which is exactly what a graph database is built for. Oracle remains the better system of record because of its strict constraints, PL/SQL and auditing. So the best design is hybrid: Oracle keeps the data correct, and Neo4j makes it searchable."

6.10 DA3 viva questions

Q: Why Neo4j and not another technology from the list?

Our data is connection-heavy (satellite → image → place → object → history), and Neo4j adds spatial and vector search in the same query language. A vector-only database would lose the relationships; a time-series database would lose the graph.

Q: How did foreign keys become relationships?

For example, image_frame.location_id became (image)-[:LOCATED_AT]->(location). The relationship itself stores the link, so no key column or join is needed.

Q: Why is OBJECT_CHANGE a relationship and not a node?

Its only purpose is to connect two objects, and its data (change %, days, rate) describes that connection, so it belongs on the edge.

Q: MERGE vs CREATE?

CREATE always adds a new node; MERGE first looks for a matching one and creates it only if missing, so re-running the load creates no duplicates.

Q: DELETE vs DETACH DELETE?

DELETE fails if the node still has relationships. DETACH DELETE removes the relationships first, then the node.

Q: What is a vector index, and what does cosine similarity measure?

A vector index finds the nearest embeddings quickly. Cosine similarity measures how closely two vectors point in the same direction: 1 means identical meaning, 0 means unrelated.

Q: How do you know Oracle and Neo4j give the same answers?

Both are loaded from the same generator. G3 returns the same 10 rows as Q10, and G8 returns the same 4 events that raise_alerts alerted.

Q: Is Neo4j replacing Oracle?

No. Oracle is the system of record; Neo4j is a query and retrieval layer on top. This is called polyglot persistence: using the right database for each job.

Part 7: The Application (Final Project)

Why it exists: the final-project brief says a report with only an ER diagram, normalization and SQL "does not constitute a complete implementation project". It requires a **working application connected to the database with at least five end-to-end features**, input validation, handling of invalid operations, duplicates, missing records and constraint violations, test cases with expected and actual results, and screenshots. We built a web app with **9 features**.

7.1 Architecture

```
  Browser  --HTTP GET/POST-->  Flask (app.py)
                                  |-- services.py : validation, GraphRAG scoring, error
translation (no DB)
                                  |-- repo.py     : every SQL statement (bind variables)
                                  |-- db.py       : connection pool, error wrapping (python-
oracledb)
                                  |-- config.py   : reads .env (user, password, DSN)
                                  |-- templates/  : Jinja2 HTML pages
                                  `-- static/     : style.css
                                         |
                                  Oracle 21c XE (JOSHWA): tables, views, pkg_eo, triggers
```

Technology	What it is	Role here
Python 3	programming language	the whole app
Flask	a small web framework	maps URLs to Python functions ("routes")
python-oracledb	Oracle's official Python driver	sends SQL to Oracle. Thin mode needs no Oracle Client install
Jinja2	a template engine	HTML files with placeholders filled from database rows
Leaflet 1.9.4	JavaScript map library	maps on the Locations, Finder and Ask pages
Chart.js 4.4.1	JavaScript chart library	tree area + NDVI chart on the location page
python-dotenv	reads .env files	keeps the password out of the code
pytest	test framework	70 automated tests

Why the code is split into layers

LAYERED ARCHITECTURE / SEPARATION OF CONCERNS Each file has **one job**. app.py handles web requests; services.py holds pure logic (testable without a database); repo.py holds all SQL (one place to review it); db.py handles connections. Changing one layer rarely breaks another, and each can be tested on its own.

7.2 What happens in one request (step by step)

Example: you open the Deforestation finder and click **Find**.

1. The browser sends `GET /finder?date_from=2026-07-01&date_to=2026-09-30&max_km=300&min_loss=10`.
2. Flask calls the `finder()` function in app.py (because of `@app.route("/finder")`).
3. `services.validate_finder()` checks that the dates are valid and from ≤ to, that 1 ≤ km ≤ 5000 and that 0 ≤ loss ≤ 100. If not, it shows a red message and uses the defaults.
4. `repo.deforestation()` runs the SQL with **bind variables** (:d_from, :d_to, :max_km, :min_loss) against v_forest_change and fn_haversine_km.
5. `db.py` borrows a connection from the **pool**, executes the query, and returns the rows as Python dictionaries.
6. app.py passes the rows to `finder.html`; Jinja builds the table, and Leaflet draws the map.
7. You see 10 events with rate, distance, trust score and alert status.

CONNECTION POOL Opening a database connection is slow (login, network handshake). The pool keeps 1–4 connections open and lends one to each request, then takes it back. Pages load faster and Oracle isn't overloaded.

7.3 The 9 features, end to end

① Dashboard (`/`)

Shows: totals (images, objects, deforestation events, open alerts), forest loss per location, open alerts and recent questions. **Database:** one SELECT with many scalar COUNT subqueries; forest loss uses the PL/SQL function `fn_forest_loss_pct`. Read-only.

② Image catalogue (`/images`, `/images/<id>`)

Shows: a filterable list (satellite, location, max cloud, dates) and a detail page (metadata, NDVI/NDWI, every detected object with model, confidence, area and attributes). **Database:** the view v_image_summary; optional filters written as `(:sat IS NULL OR satellite_name = :sat)`, so one query serves both "any" and "this one"; spectral_index; detected_object joined to type and model. **Validation:** max cloud 0–100 (test TC04). Missing image id → 404 page (TC06).

③ Location profile (`/locations`, `/locations/<id>`)

Shows: a map coloured by forest loss, then per location a **chart** of tree area and NDVI across the 5 dates, the **EVOLVED_FROM chain** (e.g. 33 → 35 → 37 → 39 → 41 for Kisangani), the change history with alert links, and the images. Novelty 1 made visible.

④ Deforestation finder (`/finder`)

The flagship question as a form (window, max km to river, minimum loss %), with a map, rate per month, distance, a **trust** score and alert status. Default result: **10 events**, confirmed by a live-database test.

⑤ Change analysis and alerts (`/alerts`)

Run change analysis calls `pkg_eo.compute_changes` and `pkg_eo.raise_alerts(min_loss, min_conf, max_cloud)`. The app reads the procedures' **DBMS_OUTPUT** (via

`dbms_output.get_line`) and shows it on the page. Each alert can be updated (status OPEN/REVIEWING/RESOLVED, assignee), which is an UPDATE. The triggers `trg_alert_resolve` (stamps resolved_at) and `trg_alert_stmt_audit` (audit row) fire automatically. Novelty 3.

⑥ Ask: GraphRAG (`/ask`, `/ask/<id>`)

1. **Validate**: question 5–500 characters; a known concept; weights ≥ 0 and not all 0; top-k 3–10; valid dates. The weights are **normalised** to sum to 1 (1, 1, 0 → 0.5, 0.5, 0).
2. **Query vector** = the centroid of labelled example embeddings for the chosen concept (repo.concept_vector).
3. **Score** every detected object in Python (services.rank_evidence): semantic (cosine), spatial ($e^{-\mathrm{km}/200}$ to the nearest river), temporal (1.0/0.3) → retrieval score; keep the top-k.
4. **One transaction**: INSERT into QUERY_LOG … `RETURNING query_id INTO :qid`; `executemany` INSERT of the evidence rows into QUERY_EVIDENCE; COMMIT. Either everything is saved or nothing.
5. **PL/SQL**: call `pkg_eo.finalize_query(qid)`, which computes and stores the answer confidence and response text.
6. **Result page**: the confidence (green ≥75%, amber 50–75%, red <50%), the response, a map of the evidence, and each evidence row with a colour bar showing its semantic / spatial / temporal contributions.

Novelties 2 and 3, live. A live test confirmed: 5 evidence rows saved, and the database's confidence equals the Python formula.

⑦ Add detection (`/detections/new`)

Form: image, type, model, confidence, area, centre lat/lon (auto-filled from the image's location). Flow: validate (confidence 0–1, area > 0, valid coordinates) → check the image exists → **block a second Tree Coverage on the same image** (duplicate check, because the lineage assumes one tree patch per image) → call **pkg_eo.add_detection** (SAVEPOINT/ROLLBACK, OUT parameter returns the new id) → `trg_obj_audit` writes the audit row.

⑧ Satellites: full CRUD (`/satellites`)

Create, Read, Update, Delete with validation (name required; launch date not in the future and not before Sputnik, 4 Oct 1957; orbit from the allowed list; revisit 1–365). Constraint errors become plain sentences:

- duplicate name → ORA-00001 → "A satellite with this name already exists."
- deleting Sentinel-2A (has images) → ORA-02292 → "Cannot delete: other records still depend on this one."
- deleting a satellite that no longer exists → "not found" (missing record).

⑨ Audit log (`/audit`)

Lists rows written **only by triggers**; the app never writes there itself. This proves the triggers work.

Also: `/health` returns `{"status":"ok","detected_objects":248}` to confirm the database connection.

7.4 Validation and error handling

Two layers of protection (defence in depth)

Layer	Where	Example
1. Application	services.py validators	"Confidence must be at most 1." is shown before the database is touched
2. Database	constraints, triggers, the package	even if someone bypasses the app, Oracle rejects bad data

The app gives friendly messages; the database guarantees correctness.

Translating Oracle errors (services.friendly_db_error)

Oracle error	Meaning	Shown to the user
ORA-00001	unique constraint violated	"A satellite with this name already exists."
ORA-02290	check constraint violated	"Cloud cover must be between 0 and 100%."
ORA-02291	parent key not found	"The selected image does not exist."
ORA-02292	child record found	"Cannot delete: other records still depend on this one."
ORA-01400	NULL into a NOT NULL column	"Required field missing: …"
ORA-20xxx	our RAISE_APPLICATION_ERROR	the message itself, e.g. "Unknown object type: Spaceship"
ORA-12541 / 12514 / 1017	cannot connect / wrong login	"Cannot connect to the database. Check that Oracle XE is running…"

The app reads the **constraint name** in the message (e.g. JOSHWA.UQ_SATELLITE_NAME) to choose the right sentence. That's one benefit of naming constraints.

The brief's four cases are all covered: **invalid input** (validators), **duplicate values** (UNIQUE + the tree-patch check), **missing records** (404 pages, "not found"), **constraint violations** (ORA codes translated).

7.5 Security

SQL INJECTION An attack where user input is pasted into SQL text and changes its meaning. If a query were built as `"… WHERE location_name = '" + input + "'"`, the input `x' OR '1'='1` would make the condition always true. **Bind variables** (`:loc`) send the input separately as pure data, so it can never become SQL. Every query in repo.py uses binds. Test TC28 proved it: the attack string returned "No images match", with no error and no leak.

- **Secrets**: the password lives in `.env`, which `.gitignore` keeps out of GitHub. Only `.env.example` (no password) is shared.
- **Data**: only synthetic data, as the brief requires.
- **Not implemented**: login/roles (a stated limitation).

7.6 How to run the app

```
cd D:\satellite-eo-knowledge-graph
pip install -r requirements.txt          (once)
copy .env.example .env                   (once; then edit with EO_DB_USER, EO_DB_PASSWORD,
EO_DB_DSN)
cd src
python app.py
# open http://127.0.0.1:5000  (keep the command window open; Ctrl+C stops the app)
```

A **virtual environment (venv)** is an optional private Python environment per project. We installed the packages into the main Python, which also works.

7.7 How the app uses DA1, DA2 and DA3

From	Used in the app
DA1	the 15 tables; total participation → required form fields; candidate keys → duplicate errors
DA2	views, fn_haversine_km, fn_forest_loss_pct, all of pkg_eo, all triggers
DA3	the same hybrid scoring as G7, implemented in Python on top of Oracle

Part 8: Testing, Results, Limitations and Future Work

8.1 Three levels of testing

Level	File	Tests	Needs Oracle?	Checks
Unit	test_services.py	~40	no	validators, cosine, haversine (Chennai–Delhi ≈ 1,757 km), ranking, the confidence formula, error translation
UI (with mocking)	test_app_smoke.py	~18	no	every page renders; every form path: valid, invalid, duplicate, blocked delete, DB down
Integration	test_integration_db.py	12	**yes**	data loaded; flagship = 10 rows; CRUD round-trip; ORA-00001/02290/02292; package error 20020; duplicate tree patch; audit trigger; resolve trigger; GraphRAG end-to-end

MOCKING Replacing a real component (here, the database layer) with a fake that returns prepared data, so the app's logic can be tested quickly and anywhere. The integration tests then confirm that the real database behaves the same way.

Result on the project laptop: `python -m pytest -v` → **70 passed in 2.79 s**. Plus **30 documented test cases** (tests/TEST_CASES.md) with expected and actual results, all **Pass**, including 4 manual browser checks (image filter, invalid input message, location profile, SQL-injection attempt).

8.2 Key results

- 36 lineage links; 10 deforestation events, of which 4 are trustworthy alerts.
- Rapid sites lost about 38% of tree area in six months (Manaus 38.72%, Tonle Sap 38.49%); slow sites about 8%.
- Answer confidences 61.2%, 66.6%, 61.8%, 60.1%, identical to the independent prediction.
- Oracle Q10 = Neo4j G3 (10 rows); Oracle alerts = Neo4j G8 (4 rows).
- NDVI–tree area correlation 0.442 (positive, as physics predicts).

8.3 Limitations

- Synthetic data and 8-dimension demo embeddings.
- Concept-based search; free-text understanding needs a language model.
- One river reference point per basin, not polygons and river lines (Oracle XE has no spatial option).
- Lineage assumes one tree patch per image per location.

- No authentication; maps and charts need internet (CDN).
- Oracle and Neo4j are loaded separately rather than kept in sync.

8.4 Future work

- Real Sentinel-2 imagery (Copernicus Data Space) with real detection models and SatCLIP-style embeddings.
- Basin polygons and river lines (PostGIS or Oracle Spatial) for true distances.
- Object matching across dates using overlap and similarity (multi-patch lineage).
- An LLM that writes the final answer from the retrieved evidence (full GraphRAG).
- Automatic Oracle → Neo4j synchronisation (change data capture).
- A benchmark of natural-language EO questions to tune α, β, γ and measure precision/recall.
- Login and role-based access.

Part 9: Viva Preparation

9.1 The 2-minute project explanation (memorise the flow, not the words)

"Our project is a **Satellite Imagery Knowledge Graph**. Satellites produce images with metadata (time, place, cloud cover) and AI detections such as buildings, ships and forest patches, but this information is usually disconnected. So questions like '*show rapid deforestation near river basins in Q3, and how much can I trust it?*' are hard to answer.

In **DA1** we designed a 15-table schema normalised to BCNF, with weak entities like SENSOR and SPECTRAL_INDEX and associative entities like QUERY_EVIDENCE and OBJECT_CHANGE.

In **DA2** we implemented it in Oracle 21c with full constraints, about 490 sample rows, 19 queries, and a PL/SQL package with triggers.

In **DA3** we extended it to Neo4j, where relationships, spatial points and vector indexes let us answer the same question as one graph query.

For the final project we built a Flask web app with 9 features on top of Oracle, tested with 70 automated tests.

Our **novelty** is threefold: we track the same forest patch over time, so change is measurable; we rank evidence by meaning, distance to the river and time with tunable weights; and we turn detection confidence and cloud cover into a trust score, so only trustworthy changes become alerts. Ten deforestation events were found, but only four had reliable enough evidence to raise an alert."

9.2 Suggested demo order (3–5 minutes)

1. Problem + architecture (1 slide).
2. ER diagram → show the 15 tables in SQL Developer.
3. Run the key query Q10 (10 rows) and Q13 (lineage chains).
4. App: Dashboard → Location profile (chart + chain) → Finder (map) → Alerts (run analysis, resolve one; show resolved_at filled) → Ask (submit, show the confidence and score bars) → Satellites (duplicate name error) → Audit log.
5. Neo4j: G3 in graph view, then G7.
6. Close: tests (70 passed) and limitations.

The brief requires **all team members to appear** in the video. Split it, for example: member 1 the problem and DA1, member 2 DA2 and the app, member 3 DA3 and the novelty.

9.3 Question bank (cover the answer, try it yourself, then check)

General

1. What problem does your project solve?

It connects fragmented satellite metadata and AI detections, so connected questions (what + where + when + how it changed + how trustworthy) can be answered.

2. Who are the users?

EO analysts (search, questions, alerts), data/ML engineers (load detections and models) and administrators (satellites, users, audit).

3. Is your data real?

No, it's synthetic, generated by a deterministic Python script with a fixed seed. The brief requires synthetic or authorised data. The same data feeds Oracle and Neo4j, so their answers can be compared.

4. What is GraphRAG?

Retrieval-Augmented Generation that retrieves evidence using a graph (connections) together with vector similarity. We implemented the retrieval and the trust scoring; LLM answer generation is future work.

5. What is an embedding?

A list of numbers produced by an AI model so that similar things have similar vectors. Ours have 8 numbers; similarity is measured with cosine.

DA1

6. Name your weak entities and their partial keys.

SENSOR (owner SATELLITE, partial key sensor_name) and SPECTRAL_INDEX (owner IMAGE_FRAME, partial key index_name).

7. If SENSOR is weak, why does it have sensor_id?

A surrogate for convenience. The identity is preserved by UNIQUE(satellite_id, sensor_name), and the existence dependency by ON DELETE CASCADE.

8. What are your associative entities?

QUERY_EVIDENCE (M:N between questions and objects, with scores) and OBJECT_CHANGE (the recursive EVOLVED_FROM with change attributes; ALERT refers to it).

9. Give an example of each cardinality in your model.

1:N, an image has many objects. 1:1, a change triggers at most one alert. M:N, questions and objects (via QUERY_EVIDENCE).

10. Give a composite, a multi-valued and a derived attribute.

Composite: bbox (4 parts). Multi-valued: embedding (a list of numbers). Derived: change_pct (from two areas).

11. Total vs partial participation, with an example.

Every image must have a location (total → NOT NULL FK). A location may or may not be in a basin (partial → nullable FK).

12. What is a functional dependency?

X → Y means equal X values always give equal Y values, e.g. satellite_name → operator_name.

13. Explain 1NF, 2NF, 3NF and BCNF in one line each.

1NF: atomic values, no repeating groups. 2NF: no dependency on part of a composite key. 3NF: no non-key → non-key dependency. BCNF: every determinant is a superkey.

14. Give a 3NF violation you removed.

image → sensor → resolution, so resolution moved to SENSOR. object → image → location → basin, so basin_id was removed from DETECTED_OBJECT.

15. Any denormalization?

Derived values in OBJECT_CHANGE, ALERT.severity, QUERY_EVIDENCE.retrieval_score/rank and QUERY_LOG.aggregated_confidence. They are historical snapshots, written only by the PL/SQL. Strictly, change_pct → change_type is a non-key dependency.

16. What are insert, update and delete anomalies?

Insert: can't add a satellite without an image. Update: changing a basin's area in hundreds of rows. Delete: deleting Chennai's last image loses its coordinates.

DA2

17. Which constraint types did you use?

PRIMARY KEY, FOREIGN KEY (CASCADE / SET NULL / restrict), UNIQUE, NOT NULL, CHECK (ranges, value lists, multi-column, IS JSON, email pattern), DEFAULT.

18. What is an identity column?

An auto-numbering column. Ours start at 10000, so app-created rows never clash with sample IDs 1–9999.

19. Why index foreign keys?

Faster joins, and Oracle avoids locking the whole child table when a parent row is deleted or updated.

20. What is a view? Name yours.

A saved query used like a table: v_image_summary (a 5-table image join) and v_forest_change (changes with location, basin, confidence and cloud).

21. What are the ACID properties?

Atomicity, consistency, isolation, durability.

22. COMMIT vs ROLLBACK vs SAVEPOINT?

Save permanently; undo since the last commit; a bookmark to roll back to partially.

23. Why did the extras' trigger survive a ROLLBACK?

DDL (CREATE TRIGGER) auto-commits.

24. INNER vs LEFT JOIN, with your example.

Q4 (inner) shows 4 satellites; E3 (left) shows Sentinel-1A with 0 images.

25. What does Q9 (MINUS) show?

Locations with deforestation but no alert (Manaus, Bhadrachalam): their evidence was too cloudy or uncertain.

26. What is a window function? Example.

A calculation over related rows without grouping them away: SUM() OVER (PARTITION BY location ORDER BY time) for cumulative loss; LAG() to get the previous patch.

27. How does Q13 (CONNECT BY) work?

START WITH the first links; CONNECT BY PRIOR curr = prev follows each chain; SYS_CONNECT_BY_PATH prints it, e.g. 1 → 3 → 5 → 7 → 9.

28. Explicit vs implicit cursor? Cursor attributes?

Implicit: created by Oracle for each DML statement (SQL%ROWCOUNT). Explicit: declared and named, with OPEN/FETCH/CLOSE. Attributes: %ISOPEN, %FOUND, %NOTFOUND, %ROWCOUNT.

29. What does WHERE CURRENT OF need?

A FOR UPDATE cursor on a single table. With a join it failed with ORA-01410.

30. Types of exceptions you used?

Predefined (NO_DATA_FOUND, DUP_VAL_ON_INDEX, VALUE_ERROR, OTHERS), user-defined (e_too_cloudy with RAISE), and RAISE_APPLICATION_ERROR (−20001… −20040).

31. What does compute_changes do?

It pairs each tree patch with the previous one at the same location (LAG), computes change %, days, rate and type, and MERGEs the result into OBJECT_CHANGE (36 links).

32. What are :OLD and :NEW?

In a row-level trigger, the row values before and after the change.

33. Which errors did you hit, and how did you fix them?

ORA-01031 (a GRANT fixed it), ORA-04098 (INSERTING moved into an IF), ORA-01410 (single-table cursor).

Novelty

34. How is "rapid" defined?

rate_pct_per_month = change_pct / days × 30; DEFORESTATION when the loss between consecutive images is ≥ 10%.

35. Write the retrieval-score formula.

α·semantic + β·spatial + γ·temporal; default 0.5/0.3/0.2; spatial = $e^{-km/200}$; temporal 1.0 or 0.3.

36. Write the answer-confidence formula.

Σ(retrieval × confidence × (1 − cloud/100)) / Σ retrieval.

37. Why store the weights with each question?

So every answer records how it was ranked, and changing the weights later doesn't silently change old answers.

38. Where does the query vector come from in the app?

The centroid of labelled examples of the chosen concept, e.g. the embeddings of DEFORESTATION patches.

DA3

39. Node, label, relationship, property?

Node = an entity instance; label = its type; relationship = a named, directed, stored link; property = a key–value pair on either.

40. How many labels and relationship types?

10 node labels and 11 relationship types; 354 nodes.

41. What is index-free adjacency?

Each node stores direct pointers to its neighbours, so traversal doesn't need index lookups or joins.

42. Which indexes did you create in Neo4j?

11 uniqueness constraints, 2 property indexes, 2 point indexes and 2 vector indexes (8-dimensional, cosine).

43. What unit does point.distance return?

Metres (hence 300000 for 300 km).

44. Why do Neo4j similarity scores differ slightly?

Neo4j normalises cosine to (1 + cos)/2.

45. Why 247 objects in Neo4j but 248 in Oracle?

The extra Oracle row is the PL/SQL demo insert, which isn't in the CSVs.

Application

46. How does the app connect to Oracle?

With python-oracledb in thin mode, through a connection pool; the credentials come from .env.

47. What is a bind variable, and why use it?

A placeholder (:id) whose value is sent separately. It prevents SQL injection and lets Oracle reuse the execution plan.

48. How do you handle a duplicate satellite name?

The UNIQUE constraint raises ORA-00001; the app maps the constraint name to "A satellite with this name already exists."

49. Show one transaction in the app.

Ask: INSERT QUERY_LOG (RETURNING id) + INSERT the evidence rows + COMMIT, then finalize_query.

50. Why split services.py and repo.py?

Logic can be unit-tested without a database, and all SQL is in one reviewable place.

51. How did you test?

Unit + UI (mocked) + live-database integration: 70 passed. Plus 30 documented test cases, all Pass.

52. Biggest limitation?

Synthetic data and demo embeddings. Next would be real Sentinel-2 imagery with real detection models and embeddings.

Glossary

ACID	Atomicity, Consistency, Isolation, Durability: the guarantees of a transaction.
Aggregate function	A function turning many rows into one value: COUNT, SUM, AVG, MIN, MAX, CORR.
Anomaly	An insert, update or delete problem caused by redundant storage.
Associative entity	A relationship promoted to an entity (QUERY_EVIDENCE, OBJECT_CHANGE).
Audit log	A record of changes; ours is written by triggers.
BCNF	Boyce–Codd Normal Form: every determinant is a superkey.
Bind variable	A placeholder (:name) whose value is sent separately from the SQL; prevents injection.
Bounding box (bbox)	The rectangle around a detected object (min/max lat/lon).
Candidate key	A minimal set of columns that uniquely identifies rows.
Cardinality	The maximum number of related instances: 1:1, 1:N, M:N.
CASCADE	An ON DELETE option that deletes child rows with the parent.
Centroid	The average of several vectors; used as a concept's query vector.
CHECK constraint	A rule a column's value must satisfy (a range or list).
CLOB	A large text column type.
Cloud cover	The percentage of an image hidden by clouds.
Confidence score	A detection model's certainty, 0–1.
CONNECT BY	Oracle's hierarchical (tree/chain) query syntax.
Connection pool	A set of open database connections reused across requests.
Correlated subquery	A subquery that refers to the outer query's current row.
Cosine similarity	The similarity of two vectors by angle: 1 same, 0 unrelated.
CRUD	Create, Read, Update, Delete.
Cursor	A pointer to query result rows, processed one by one in PL/SQL.
Cypher	Neo4j's query language.
Data dictionary	Oracle's catalogue tables describing your objects (USER_TABLES…).
DCL / DDL / DML / TCL	Control (GRANT) / Definition (CREATE) / Manipulation (INSERT) / Transaction (COMMIT) statements.
Denormalization	Deliberately storing redundant or derived data, for a stated reason.
Derived attribute	A value computable from other data (change_pct).
DETACH DELETE	A Cypher delete that also removes the node's relationships.
Determinant	The left side X of an FD X → Y.

DSN	Data Source Name: the connection address, e.g. localhost:1521/XEPDB1.
Embedding	A vector of numbers representing meaning.
Entity	A real-world thing stored in the database.
EO	Earth Observation.
EVOLVED_FROM	Our link from an object to its earlier observation (temporal lineage).
Exception	A PL/SQL error condition that can be caught and handled.
Flask	A Python web framework.
Foreign key	A column referencing another table's primary key.
Functional dependency	X → Y: X determines Y.
GraphRAG	Retrieval-augmented generation that retrieves through a graph + similarity.
Haversine	The formula for distance on a sphere.
HAVING	Filters groups after GROUP BY.
Identity column	An auto-numbered column.
Index	A structure for finding rows quickly without a full scan.
Integration test	A test using the real database.
JSON	A text format for structured data; checked with IS JSON.
Label (Neo4j)	A node's type.
LAG()	A window function returning the previous row's value.
LOAD CSV	The Cypher command to import CSV files.
Lossless decomposition	A split whose pieces join back to exactly the original.
MERGE	Insert-or-update (SQL), match-or-create (Cypher).
Mocking	Replacing a real component with a fake in tests.
NDVI / NDWI	Vegetation / water spectral indices (−1 to 1).
Node	A thing in a graph database.
Normalization	Organising tables to remove redundancy (1NF → BCNF).
Package	A named group of PL/SQL procedures and functions (spec + body).
Partial key	A weak entity's key, unique only within its owner.
Participation	Whether every instance must take part in a relationship (total/partial).
PL/SQL	Oracle's procedural language around SQL.
Primary key	The chosen unique, non-null identifier of rows.
Property (Neo4j)	A key–value pair on a node or relationship.
RAISE_APPLICATION_ERROR	Raises a custom numbered error (−20000…−20999).
Recursive relationship	An entity related to itself (object → earlier object).

Relationship	An association between entities (ER) or a stored link between nodes (graph).
River basin	The land draining into one river.
ROLLBACK / SAVEPOINT	Undo changes / a bookmark for partial undo.
Route	A URL handled by a Flask function.
Schema	A database's structure; in Oracle also the objects owned by a user.
SET NULL	An ON DELETE option that empties the child's FK.
Spatial resolution	The ground size of one pixel.
SQL injection	An attack inserting SQL through user input; prevented by binds.
Superkey	Any set of columns that uniquely identifies rows.
Surrogate key	An artificial numeric primary key.
Synthetic data	Realistic but generated data.
Transaction	Changes that succeed or fail together.
Transitive dependency	X → Z → Y through a non-key Z (violates 3NF).
Trigger	PL/SQL that runs automatically on INSERT/UPDATE/DELETE.
Unit test	A test of one small piece of logic in isolation.
Vector index	An index for fast nearest-embedding search.
View	A stored query used like a table.
Weak entity	An entity identified only via its owner + a partial key.
Window function	A calculation over related rows without collapsing them (OVER clause).
XEPDB1	The pluggable database in Oracle XE where our schema lives.